

Peacehealth Bikes Important Papers

Authors: Drew Moulton, Peyton Phillips, Dacian Rapolla, Jack Sedillos

Revisions History

Date modified	Modified by	Key additions
Thu May 14 2026	Peyton Phillips	Document creation
Fri May 15 2026	Peyton Phillips	SRS, SDS first draft made (too long needs to be short)
Fri May 15 2026	Drew Moulton	Condense SRS/SDS/plan to one page each
Fri May 15 2026	Dacian Rapolla	Add project timeline
Tue May 19 2026	Peyton Phillips	Long Draft started; sections formatted; SRS draft
Wed May 20 2026	Drew Moulton	Add Gantt chart and project plan
Wed May 20 2026	Peyton Phillips	Add SDS draft; Add diagrams
Thu May 21 2026	Dacian Rapolla	Fixed and finalized SRS/SDS
Mon Jun 1 2026	Peyton Phillips	Revisions to SRS/SDS/Project Proposal

Revisions History.....	0
SRS.....	7
Concept of Operations.....	7
Current System or Situation.....	7
Justification for a New System.....	7
Operational Features of the Proposed System.....	8
User Classes.....	10
Occasional Bike-Share Users.....	10
University of Oregon Students, Faculty, and Staff.....	10
Transit or Last-Mile Transportation Users.....	10
System Evaluators and Installers.....	11
Developers and Maintainers.....	11
Modes of Operation.....	11
Normal Live Data Mode.....	11
Demo/Sample Data Mode.....	11
Degraded Data Mode.....	11
Installation/Test Mode.....	11
Operational Scenarios / Use Cases.....	12
Case 1: Find the Nearest Available Bike.....	12
Brief Description.....	12
Actors.....	12
Preconditions.....	12
Steps to Complete the Task.....	12
Postconditions.....	12
Case 2: Find a Hub With Bikes Available.....	13
Brief Description.....	13
Actors.....	13
Preconditions.....	13
Steps to Complete the Task.....	13
Postconditions.....	13
Case 3: Use the System When Live Data Is Temporarily Unavailable.....	13
Brief Description.....	13
Actors.....	13
Preconditions.....	14
Steps to Complete the Task.....	14
Postconditions.....	14
Case 4: View System Alerts.....	14

Brief Description.....	14
Actors.....	14
Preconditions.....	14
Steps to Complete the Task.....	14
Postconditions.....	15
Specific Requirements.....	15
External Interfaces: Inputs and Outputs.....	15
Public GBFS Feed Input.....	15
Name:.....	15
Purpose:.....	15
Source:.....	15
Data format:.....	15
Expected content:.....	15
Valid Input Range:.....	15
Invalid input handling:.....	15
User Starting Location Input.....	16
Name:.....	16
Purpose:.....	16
Source:.....	16
Data format:.....	16
Valid range:.....	16
Invalid input handling:.....	16
Map Marker Selection Input.....	16
Name:.....	16
Purpose:.....	16
Source:.....	16
Data format:.....	16
Valid range:.....	16
Invalid input handling:.....	16
Bike and Hub Map Output.....	17
Name:.....	17
Purpose:.....	17
Destination:.....	17
Data format:.....	17
Valid output:.....	17
Directions Output.....	17
Name:.....	17
Purpose:.....	17

Destination:.....	17
Data format:.....	17
Valid output:.....	17
System Status Output.....	17
Name:.....	17
Purpose:.....	18
Destination:.....	18
Data format:.....	18
Valid output.....	18
Functional Requirements.....	18
FR-1. Display Map.....	18
FR-2. Load public bike-share data.....	18
FR-3. Display available bikes.....	18
FR-4. Display hubs or stations.....	18
FR-5. Display hub availability.....	18
FR-6. Distinguish bikes from hubs.....	18
FR-7. Show data freshness.....	18
FR-8. Accept starting location.....	19
FR-9. Select Destination.....	19
FR-10. Compute nearby options.....	19
FR-11. Sort or highlight nearby results.....	19
FR-12. Provide route or direction information.....	19
FR-13. Display feed errors.....	19
FR-14. Provide sample data mode.....	19
FR-15. Avoid account features.....	19
FR-16. Avoid rental actions.....	19
FR-17. Display official-app handoff message.....	19
FR-18. Provide health/status endpoint or panel.....	19
FR-19. Filter marker types.....	20
FR-20. Display alerts.....	20
FR-21. Show a nearby list view.....	20
FR-22. Cache last successful data.....	20
FR-23. Show service area information.....	20
FR-24. Provide installation test command.....	20
Usability Requirements.....	20
UR-1. First-use clarity.....	20
UR-2. Marker clarity.....	20
UR-3. Selection feedback.....	20

UR-4. Error clarity.....	20
UR-5. Data freshness visibility.....	21
UR-6. Mobile usability.....	21
UR-7. Minimal interaction path.....	21
UR-8. Official-app separation.....	21
Performance Requirements.....	21
PR-1. Initial page load.....	21
PR-2. Sample data load.....	21
PR-3. Backend API response time.....	21
PR-4. Distance computation.....	21
PR-5. Error response time.....	21
PR-6. Browser support.....	22
Software System Attributes.....	22
Reliability.....	22
Maintainability.....	22
Privacy.....	22
Portability.....	22
Testability.....	22
Transparency.....	22
Security.....	23
Requirements Elicitation Plan.....	23
Passerby Student (1):.....	23
Q1. Have you used PeaceHealth Rides before? Why or why not?.....	23
Q2. What information would you want before deciding to walk to a particular bike or hub?.....	23
Q3. Is there any information you would want as you are walking to the bike or hub?...	23
Q4. Would you prefer a map, a list of nearby bikes, or both?.....	23
Q5. What is a good time frame to update bike data in your opinion?.....	23
Q6. What error messages or warnings would be helpful if bike data is unavailable?..	23
General Findings.....	24
SDS.....	25
System Overview.....	25
Software Architecture.....	25
Frontend Architecture Planning.....	26
Bike Finder Map.....	26
Leaflet.....	26
OpenStreetMap.....	26
Frontend Responsibilities.....	26

Frontend Architecture Rationale.....	26
Backend Architecture Planning.....	27
Ride Feed Collector.....	27
Ride Data Normalizer.....	27
Availability Query Service.....	27
Route Guidance Service.....	27
OSRM (Open Source Routing Machine).....	28
Status and Fallback Service.....	28
Dashboard API.....	28
Local Demonstration Data Store.....	28
Backend Responsibilities.....	28
Backend Architecture Rationale.....	29
Technology Choices.....	29
Alternative Designs Considered.....	30
Alt-1: Direct Browser-to-GBFS design.....	30
Alt-2: Full Database-driven design.....	30
Alt-3: Static data only design.....	30
Software Modules.....	30
Ride Feed Collector.....	30
Role and Primary Function.....	30
Interface to Other Modules.....	31
Static Model.....	31
Dynamic Model.....	32
Design Rationale.....	32
Ride Data Normalizer.....	32
Role and Primary Function.....	32
Interface to Other Modules.....	32
Internal Data Objects.....	33
Static Model.....	33
Dynamic Model.....	34
Design Rationale.....	34
Availability Query Service.....	34
Role and Primary Function.....	34
Interface to Other Modules.....	34
Static Model.....	35
Dynamic Model.....	35
Design Rationale.....	36
Route Guidance Service.....	36
Role and Primary Function.....	36

Interface to Other Modules.....	36
Static Model.....	37
Dynamic Model.....	37
Design Rationale.....	38
Status and Fallback Service.....	38
Role and Primary Function.....	38
Interface to Other Modules.....	38
Static Model.....	39
Dynamic Model.....	39
Design Rationale.....	40
Dashboard API.....	40
Role and Primary Function.....	40
Interface to Other Modules.....	40
Static Model.....	41
Dynamic Model.....	41
Design Rationale.....	42
Bike Finder Map.....	43
Role and Primary Function.....	43
Interface to Other Modules.....	43
Static Model.....	43
Dynamic Model.....	44
Design Rationale.....	44
Local Demonstration Data Store.....	44
Role and Primary Function.....	44
Interface to Other Modules.....	45
Static Model.....	45
Dynamic Model.....	45
Design Rationale.....	45
Dynamic Models of Operational Scenarios.....	45
Project Plan.....	46
Planning and Architecture.....	46
Core Structure Production.....	47
Parallel Feature Development.....	47
Directions and Routing.....	47
Integration and Testing.....	48
Documentation.....	48
References.....	49
Acknowledgements.....	50

SRS

Concept of Operations

This Software Requirements Specification describes the expected behavior, user interactions, external interfaces, and measurable requirements for the PeaceHealth Rides Availability and Navigation Dashboard. The SRS follows the course template structure, which includes a Concept of Operations, user classes, operational modes, operational scenarios, and specific requirements.

Current System or Situation

PeaceHealth Rides is a public bike-share system in Eugene and Springfield. The City of Eugene describes PeaceHealth Rides as a network of publicly available bikes that users can pick up and drop off around downtown, mid-town, up-town, and Springfield's downtown station area. The City of Eugene page currently describes the system as having 40 bike share stations and 300 bikes [1].

PeaceHealth Rides also has an official app and website. The official PeaceHealth Rides website describes the process of scanning a bike's QR code to unlock it, riding the bike, and returning it by locking it at a hub or public bike rack [2]. The App Store listing says the official app lets users find bicycles on a map, create an account, pay through the app, reserve bicycles wirelessly, manage an account, and contact customer support [3].

The University of Oregon is a realistic stakeholder community for this system because UO Transportation Services advertises PeaceHealth Rides pricing options for students, faculty, and staff using a UO email address [4].

PeaceHealth Rides also publishes public bike-share availability data through the General Bikeshare Feed Specification, or GBFS. GBFS is an open data standard for shared mobility systems, and its documentation describes the types of files and fields used in bike-share feeds [5]. The GBFS project describes itself as an open data standard that helps people discover and use shared mobility modes [6].

The current system therefore already provides official functions for renting, reserving, unlocking, and paying for bikes. However, those functions are tied to the official PeaceHealth Rides app and account workflow. This project does not attempt to replace those official functions. Instead, it proposes a narrower public-information tool that helps users view bike and hub availability and navigate toward available bikes or hubs without requiring a PeaceHealth Rides login.

Justification for a New System

The proposed system is justified as a lightweight public-information dashboard rather than a replacement for the official PeaceHealth Rides app. The official app supports account creation,

payment, wireless bicycle reservation, account management, and support contact features [3]. Those functions are necessary for actually renting a bike, but they are more than some users need when they are only deciding whether there is a bike nearby. A user may want to answer a smaller set of questions before opening the official app or creating an account:

- “Is there a bike near me right now?”
- “Which nearby hub has bikes available?”
- “How far would I have to walk to reach the closest bike or hub?”
- “Is the system feed working, and how recently was the information updated?”

The need for the system was supported by 33 interviews with potential users. Interviewees repeatedly described three practical problems: deciding whether a bike is close enough to walk to, trusting whether displayed bike availability is current, and choosing between map-based browsing and a faster nearby list. These findings changed the SRS by emphasizing nearby bike ranking, data freshness labels, fallback warnings, and a ranked nearby list. The system is therefore not based only on the team’s assumptions; it is based on recurring transportation-planning concerns reported by potential users.

The proposed system addresses this narrower need by displaying public availability data and directions in a browser-based interface. This project is appropriate for the course because it can be divided into modules for parallel development, such as data fetching, data normalization, API development, map display, routing/directions, testing, and documentation. The project handout emphasizes that projects should be decomposable into modules for parallel development and should be based on realistic requirements from real users [7]. The system also fits the course’s installation constraints. The project rules state that the delivered system should not require software beyond a Python virtual environment or Docker objects, should not require the evaluator to create accounts on online services, and should include installation instructions in the README [7]. Because the proposed system uses public availability data and does not require PeaceHealth Rides login credentials, it can be evaluated without an external user account.

Operational Features of the Proposed System

The proposed system is a browser-based dashboard that displays PeaceHealth Rides bike and hub availability on a map. The system will retrieve public GBFS data, transform that data into a simplified internal representation, and display bikes, hubs, and availability information to the user.

The system will provide the following high-level operational features:

1. Display a map of the PeaceHealth Rides service area

Justification:

This gives users a visual way to understand where bikes and hubs are located instead of reading availability information as a list of coordinates or station names.

2. Display available individual bikes from public availability data

Justification:

This is the core feature of the system because the user needs to know which bikes are currently available before walking toward one.

3. Display hubs or stations from public station data

Justification:

Hubs and stations provide fixed reference points for users and help organize bike availability around known locations in the service area.

4. Display availability information for hubs, such as available bikes, if provided by the feed

Justification:

This helps users compare destinations before choosing where to go. A hub with more available bikes may be a safer destination than a location with only one visible bike.

5. Show when the displayed availability data was last updated

Justification:

Bike availability can change quickly. Showing the update time helps users understand whether the displayed information is current or potentially stale.

6. Allow a user to enter or select a starting location

Justification:

The system needs a starting point in order to estimate distance or directions to a selected bike or hub.

7. Allow a user to select a bike or hub destination

Justification:

Selecting a destination lets the system focus on one bike or hub and provide specific route or distance information instead of only showing general map data.

8. Provide walking directions or a walking route estimate to the selected bike or hub

Justification:

This directly supports the goal of helping users get to an available bike or hub, not just see where bikes are located.

9. Display public system alerts when available

Justification:

Alerts may contain important service information, such as outages, closures, or other conditions that could affect whether a bike or hub is usable.

10. Provide readable error messages when live data cannot be loaded

Justification:

If the live feed fails, the system should explain the issue clearly instead of leaving the map blank or confusing the user.

11. Provide sample data mode for demonstrations and testing

Justification:

Sample data allows the team to test and demonstrate the system even when live network access or public feed data is unavailable.

12. Avoid login, payment, reservation, unlocking, or account management features

Justification:

These features are outside the scope of the project. The system is meant to help users find and navigate to bikes or hubs, not replace the official PeaceHealth Rides app.

The system will not allow users to rent or unlock bicycles. It will direct users to the official PeaceHealth Rides app or website when they need official rental features such as account login, payment, reservation, or unlocking.

User Classes

Occasional Bike-Share Users

Occasional bike-share users are people who may use PeaceHealth Rides sometimes but do not necessarily commute with it every day. They may include visitors, downtown users, or people trying bike share for errands or short trips. These users expect the system to be simple, fast, and understandable without training. They may not know where the nearest hub is, and they may not want to open the official app until they know a bike is nearby.

University of Oregon Students, Faculty, and Staff

UO students, faculty, and staff are a realistic user class because UO Transportation Services advertises PeaceHealth Rides plans for the university community [4]. These users may use bike share to travel between campus, housing, downtown Eugene, and transit stops. They are likely familiar with web maps and mobile apps, but they may not know the current location or availability of nearby bikes.

Transit or Last-Mile Transportation Users

Transit or last-mile users are people who combine walking, bus travel, and bike share to complete trips. They may want to know whether a bike is available near a bus stop or whether there is a hub near their destination. These users expect clear location information and quick answers because their trip decisions may depend on timing.

System Evaluators and Installers

System evaluators and installers include the course instructor, graders, and teammates who run the project locally. These users expect the system to install cleanly, run without external account creation, and demonstrate its main features with either live public data or provided sample data. The course project rules require installation instructions and prohibit requiring the evaluator to create accounts on online services [7].

Developers and Maintainers

Developers and maintainers include team members who extend, test, or debug the system. They expect the codebase to be modular, documented, and testable. They need clear error messages when the GBFS feed is unavailable or when data fields are missing.

Modes of Operation

Normal Live Data Mode

In normal live data mode, the system retrieves current public GBFS data, displays available bikes and hubs, and provides directions or distance information based on the user's selected starting location.

Demo/Sample Data Mode

In demo/sample data mode, the system uses local sample JSON files instead of live network data. This mode allows the project to be demonstrated when internet access is unavailable or when the live feed is temporarily unreachable. The course handout encourages realistic sample data and installation designs that make the system easy to see working [7].

Degraded Data Mode

In degraded data mode, some external data is unavailable, stale, or malformed. The system should still show any previously cached or sample availability data that is safe to display, and it should clearly tell the user that the live feed could not be refreshed.

Installation/Test Mode

In installation/test mode, the system runs checks that confirm the backend, frontend, data parser, sample data, and external feed access are working. This mode is intended for evaluators and developers.

Operational Scenarios / Use Cases

Case 1: Find the Nearest Available Bike

Brief Description

A user wants to quickly find the closest available PeaceHealth Rides bike near their current or entered location

Actors

Occasional bike-share user, UO student/faculty/staff user, transit/last-mile user

Preconditions

1. The user has opened the web application in a browser
2. The system has loaded the map interface
3. The system has access to live GBFS data or local sample data
4. The user has either allowed location access or entered a starting location manually

Steps to Complete the Task

1. The user opens the PeaceHealth Rides Availability and Navigation Dashboard
2. The system loads the map and requests bike availability data
3. The system displays available bikes as markers on the map
4. The user enters a starting address, clicks a point on the map, or uses browser geolocation
5. The system calculates distance from the starting location to nearby available bikes
6. The system highlights or lists the nearest available bikes
7. The user selects one bike
8. The system displays the selected bike's location, approximate distance, and last data update time
9. The system displays walking directions or a walking route estimate to the bike
10. The user follows the directions to the bike
11. The user uses the official PeaceHealth Rides app or approved unlock methods if they choose to rent the bike

Postconditions

1. The user has identified a nearby available bike
2. The user has directions or distance information to reach the bike

3. The system has not required the user to log into PeaceHealth Rides
4. The system has not reserved, unlocked, or rented the bike

Case 2: Find a Hub With Bikes Available

Brief Description

A user wants to find a nearby PeaceHealth Rides hub or station with bikes available

Actors

UO user, transit / last-mile user, occasional bike-share user

Preconditions

1. The user has opened the web application
2. The system has loaded hub/station data
3. The system has loaded station availability data if available from the public feed
4. The user has selected or entered a starting location

Steps to Complete the Task

1. The user opens the dashboard
2. The system displays PeaceHealth Rides hubs on the map
3. The user switches the map view or filter to show hubs
4. The system displays hub markers with availability indicators
5. The user enters a starting location or selects their current location
6. The system sorts or highlights nearby hubs based on distance and bike availability
7. The user selects a hub marker
8. The system displays the hub name or identifier, approximate distance, number of available bikes if provided, and last update time
9. The system displays directions or a walking route estimate to the hub
10. The user uses the directions to reach the hub

Postconditions

1. The user has identified a nearby hub with bike availability information
2. The user has directions or distance information to reach that hub
3. The system has displayed the age or update time of the availability data

Case 3: Use the System When Live Data Is Temporarily Unavailable

Brief Description

A user or evaluator opens the system when the live GBFS feed cannot be reached, and the system provides a useful fallback instead of failing silently.

Actors

Occasional user, evaluator, developer

Preconditions

1. The user has opened the web application
2. The live GBFS feed is unavailable, blocked, malformed, or unreachable
3. The system has local sample data or cached data available

Steps to Complete the Task

1. The user opens the dashboard
2. The frontend requests bike and hub data from the backend
3. The backend attempts to retrieve or refresh the public feed
4. The feed request fails or returns invalid data
5. The backend records the failure and returns a structured error status
6. The frontend displays a visible message explaining that live data could not be refreshed
7. If sample or cached data is available, the system displays it with a clear label such as “Sample Data” or “Cached Data”
8. The user can still inspect the amp, markers, and general interaction flow
9. The user can view the system status panel to see which components are working

Postconditions

1. The system does not crash
2. The user understands that the displayed information may not represent current bike availability
3. The evaluator can still test the interface using sample or cached data

Case 4: View System Alerts

Brief Description

A user checks whether there are public bike-share system alerts before relying on the service.

Actors

Bike-share user, transit/last-mile user, UO user

Preconditions

1. The system has loaded public GBFS feed information
2. Public system alert information is available or the system can determine that no alerts are currently provided

Steps to Complete the Task

1. The user opens the dashboard

2. The system checks the public alerts feed if available
3. The system displays an alert banner or status area
4. If alerts exist, the system displays the alert title, affected area or station if available, and alert description
5. If no alerts exist, the system displays “No current alerts available” or leaves the alert panel inactive
6. The user uses the alert information to decide whether to continue planning a bike-share trip

Postconditions

1. The user has seen relevant public alert information if available
2. The user understands whether the system is reporting alerts

Specific Requirements

External Interfaces: Inputs and Outputs

Public GBFS Feed Input

Name:

Peacehealth Rides public GBFS feed

Purpose:

Provide public bike-share system data

Source:

PeaceHealth Rides public GBFS endpoint discovered through the GBFS auto-discovery feed

Data format:

JSON

Expected content:

Bike locations, station/hub information, station availability, system information, and alerts when available. GBFS documentation defines the files and fields used in shared mobility feeds [5]

Valid Input Range:

The system shall accept valid JSON objects that follow the expected GBFS structure

Invalid input handling:

The system shall reject malformed JSON, missing required fields, or unsupported feed structures and shall report a readable error

User Starting Location Input

Name:

User starting location

Purpose:

Determine distance and directions from the user to a bike or hub.

Source:

Browser geolocation, typed address, typed coordinates, or map click.

Data format:

Latitude/longitude pair or address string

Valid range:

Latitude must be between -90 and 90. Longitude must be between -180 and 180.

Invalid input handling:

The system shall display an error if the location is empty, outside valid coordinate bounds, or cannot be resolved

Map Marker Selection Input

Name:

Destination selection

Purpose:

Allow the user to choose a bike or hub

Source:

User click or tap on a map marker or list item

Data format:

Internal bike ID, station ID, or marker ID

Valid range:

The selected ID must match a currently displayed bike or hub

Invalid input handling:

The system shall ignore invalid marker IDs and display a non-crashing error message if necessary

Bike and Hub Map Output

Name:

Availability map

Purpose:

Display available bikes and hubs visually

Destination:

User's browser

Data format:

HTML, CSS, JavaScript-rendered map markers

Valid output:

Map markers should distinguish bikes from hubs

Directions Output

Name:

Walking directions or route estimate

Purpose:

Helps users navigate to a selected bike or hub

Destination:

User's browser

Data format:

Route line, step list, distance estimate, or direct walking-distance estimate depending on implementation

Valid output:

Directions shall identify a selected destination and show distance in miles, feet, meters, or kilometers.

System Status Output

Name:

System status panel or message

Purpose:

Tell users and evaluators whether live data, sample data, cached data, and core services are working

Destination:

User's browser

Data format:

Human-readable status message and/or JSON health endpoint

Valid output

The system shall clearly label live, cached, or sample data

Functional Requirements

FR-1. Display Map

The system shall display a browser-based map centered on the PeaceHealth Rides service area

FR-2. Load public bike-share data

The system shall retrieve or load PeaceHealth Rides bike-share data from public GBFS JSON or provided sample JSON

FR-3. Display available bikes

The system shall display available bikes as map markers when bike location data is available

FR-4. Display hubs or stations

The system shall display PeaceHealth Rides hubs or stations as map markers when station data is available

FR-5. Display hub availability

The system shall display the number of available bikes at a hub when that information is available from the data source

FR-6. Distinguish bikes from hubs

The system shall visibly distinguish individual bike markers from hub or station markers

FR-7. Show data freshness

The system shall display the last update time or feed timestamp for bike and hub data when the source provides it

FR-8. Accept starting location

The system shall allow the user to provide a starting location by at least one method, such as manual coordinate entry, address entry, map click, or browser geolocation

FR-9. Select Destination

The system shall allow the user to select a bike or hub as a destination

FR-10. Compute nearby options

The system shall compute approximate distance from the user's starting location to displayed bikes or hubs

FR-11. Sort or highlight nearby results

The system shall identify at least the nearest available bike or hub from a valid starting location

FR-12. Provide route or direction information

The system shall provide walking directions, a route line, or an approximate walking-distance estimate from the user's starting location to the selected bike or hub

FR-13. Display feed errors

The system shall display a readable error message if live data cannot be loaded

FR-14. Provide sample data mode

The system shall include local sample data so the main interface can be demonstrated without live feed access

FR-15. Avoid account features

The system shall not require users to create or use a PeaceHealth Rides account

FR-16. Avoid rental actions

The system shall not attempt to reserve, rent, unlock, pay for, or end a PeaceHealth Rides trip

FR-17. Display official-app handoff message

The system shall tell users to use the official PeaceHealth Rides app or approved unlock method for account, payment, reservation, and unlocking actions

FR-18. Provide health/status endpoint or panel

The system shall provide a way for evaluators to see whether the data source, backend, frontend, and sample data are functioning.

FR-19. Filter marker types

The system should allow users to show bikes only, hubs only, or both

FR-20. Display alerts

The system should display public system alerts when available from the GBFS feed

FR-21. Show a nearby list view

The system should display a ranked list of nearby bikes or hubs in addition to map markers

FR-22. Cache last successful data

The system should cache the last successful live feed response and label it as cached when live refresh fails

FR-23. Show service area information

The system should display the PeaceHealth Rides service area or a note when a selected starting point appears outside the expected area.

FR-24. Provide installation test command

The system should provide a documented command that checks whether the backend, frontend, and sample data are working.

Usability Requirements

UR-1. First-use clarity

The main page shall display the title “PeaceHealth Rides Availability and Navigation Dashboard” and a one-sentence description stating that the system helps users find available PeaceHealth Rides bikes and hubs. In a usability check with at least 5 first-time users, at least 4 users shall correctly describe the system’s purpose as finding available bikes/hubs or directions to bikes/hubs within 30 seconds of viewing the main page.

UR-2. Marker clarity

Bike markers and hub markers shall use visually distinct marker styles. A bike marker and a hub marker shall differ by at least two of the following: icon, label text, marker color, or marker shape.

UR-3. Selection feedback

When a user selects a bike or hub marker, the system shall display a destination details panel within 1 second. The panel shall include destination type, distance or route estimate when available, and data freshness information when available.

UR-4. Error clarity

When live data cannot be loaded, the system shall display one of the following error categories: live feed unavailable, invalid feed data, missing sample data, invalid user location, or route unavailable. The message shall be visible without opening browser developer tools.

UR-5. Data freshness visibility

The system shall display one of the following labels whenever bike or hub data is shown: “Live Data,” “Cached Data,” or “Sample Data.” If cached or sample data is displayed, the system shall include a warning that the data may not represent current bike availability.

UR-6. Mobile viewport layout

At viewport sizes of 390x844 pixels and 360x640 pixels, the main map, location input, data status label, and selected destination panel shall be accessible without horizontal scrolling. The user shall be able to select a bike or hub marker and view its content at those viewport sizes.

UR-7. Minimal interaction path

Starting from the loaded main page in sample data mode, a user shall be able to view route or distance information for a nearby bike in no more than four primary actions: provide or select starting location, choose bike or hub mode if needed, select a result, and open route or distance details.

UR-8. Official-app separation

The main page or destination details panel shall include the text “This dashboard does not unlock, reserve, rent, or pay for bikes. Use the official PeaceHealth Rides app for rental actions.”

Performance Requirements

PR-1. Initial page load

In sample data mode, after running ‘docker compose up –build’, the frontend shall display the map container, status label, and at least one sample bike or hub marker within 15 seconds of opening ‘http://localhost:8080’ on the evaluator’s machine

PR-2. Sample data load

In sample data mode, the system shall load sample bike and hub markers within 3 seconds after the page is opened on the evaluators machine

PR-3. Backend API response time

For local sample data requests, 95% of backend API responses shall complete in under 1 second on the evaluator’s machine

PR-4. Distance computation

For up to 500 displayed bikes and hubs, the system shall compute approximate distances from a selected starting point in under 1 second on the evaluator’s machine

PR-5. Error response time

If live feed retrieval fails, the system shall display an error or fallback state within 10 seconds of the failed request

PR-6. Browser support

The system shall run in the latest stable versions of Chrome and Firefox without requiring browser extensions, native app installation, or user login.

Software System Attributes

Reliability

The system shall avoid failing silently. If live data cannot be loaded, the user shall see an explanation and, when possible, sample or cached data. This matters because users may make transportation decisions based on availability information.

Maintainability

The system shall separate feed retrieval, data normalization, backend API routes, map display, directions, and status reporting into modules. This supports parallel development and makes the system easier to debug.

Privacy

The system shall not collect PeaceHealth Rides account credentials, payment information, trip history, or private user account data. If browser geolocation is used, the system should use it only for the current session unless the user explicitly saves a location locally.

Portability

The system shall be installable using Docker. The course project rules state that the delivered system should not require native software installation beyond approved environments such as Python venv or Docker [9]

Testability

The system shall include sample JSON data and tests for data parsing, distance calculation, API response formatting, and error handling. The system should be testable without requiring live PeaceHealth Rides data.

Transparency

The system shall clearly label whether it is displaying live, cached, or sample data. This matters because bike availability changes over time, and users should not mistake stale or sample data for current availability.

Security

The system shall not store secrets, API keys, passwords, or user credentials. The system shall validate external JSON before using it and shall avoid displaying raw untrusted error traces to public users.

Requirements Elicitation Plan

The interviews with students and parents were conducted informally using a short set of open-ended questions. The goal was to understand how potential users decide whether to use PeaceHealth Rides, what information they need before walking to a bike or hub, and what would make a public availability dashboard useful or confusing. Below is 1 of 33 interviews conducted, and the other interviews are in another document within the project.

Passerby Student (1):

Q1. Have you used PeaceHealth Rides before? Why or why not?

I've used it before. I need it to get to campus from my apartment.

Q2. What information would you want before deciding to walk to a particular bike or hub?

I mainly walk to the same hub outside my apartment. If I'm leaving campus though, I'll try to use the same bike. Sometimes I can't find it though and end up looking around or walking.

Q3. Is there any information you would want as you are walking to the bike or hub?

I'd want it to be on the way to where I'm trying to go, you know? Don't want to waste my time going across campus to get a bike.

Q4. Would you prefer a map, a list of nearby bikes, or both?

Probably a list of nearby bikes, since my main issue is just finding the bikes when I'm leaving.

Q5. What is a good time frame to update bike data in your opinion?

I mean I'd like it to be as soon as possible. When I'm leaving class I have to get back in time to eat dinner and study, so saving as much time as possible would be nice.

Q6. What error messages or warnings would be helpful if bike data is unavailable?

If the data isn't there isn't the app just not functioning? Probably a warning on that.

General Findings

Participant group	Number Interviewed	Relevance to Project
Passerby Students	24	Students are a major potential user group because they may use bikes to travel between campus, housing, jobs, grocery stores, labs, and recreation areas.
Passerby Parents	9	Parents and visitors represent less frequent users who may need clearer directions, safer route information, and more explanation than daily users.
Total	33	The interviews provide broader support for the project requirements than the original single-interview draft

Finding	Number of participants	Requirement impact
Preferred a map view or both map and list view	25 / 33	Supports FR-3, FR-4, FR-6, FR-19, and the Bike Finder Map module
Preferred a list view or both map and list view	16 / 33	Supports FR-21
Wanted bike/hub data to update in 5 minutes or less	25 / 33	Supports FR-7, UR-5, PR-5, and the Status and Fallback Service.
Wanted a warning when data is unavailable	33 / 33	Supports FR-8, FR-10, FR-11, FR-12, and the Route Guidance Service.
Mentioned distance, walking time, route safety, construction, destination hub availability, or nearby navigation	24 / 33	Supports FR-8, FR-10, FR-11, FR-12, and the Route Guidance Service.
Mentioned bike condition, safety, cleanliness, adjustability, lights, brakes, or maintenance information	14 / 33	Supports future enhancement requirements, but most of these details may not be available from the public GBFS feed.
Mentioned official rental concerns such as unlocking, reservations, payment, or pricing	8/33	Supports FR-15, FR-16, FR-17, and UR-8, because the dashboard should clearly explain that rental actions remain part of the official PeaceHealth Rides app.

Requirement Changes Based on Interviews

The interview results strengthened the justification for the system. The original SRS assumed that users would benefit from a lightweight public dashboard, but the interviews showed recurring real-world needs. Participants repeatedly wanted to avoid wasting time walking to a bike that was unavailable, too far away, or not on the way to their destination. This supports requirements for nearest-bike lookup, distance calculation, route guidance, and visible data freshness.

The interviews also showed that users do not all want the same interface. Many participants preferred a map because it helps them understand location and direction, while others preferred a list because it is faster to scan nearby options. This supports keeping both map markers and a ranked nearby list in the requirements.

Finally, every participant gave some form of expected warning or message for unavailable data. This supports the system's degraded data mode, status banner, fallback behavior, and requirement that live, cached, and sample data be clearly labeled.

SDS

System Overview

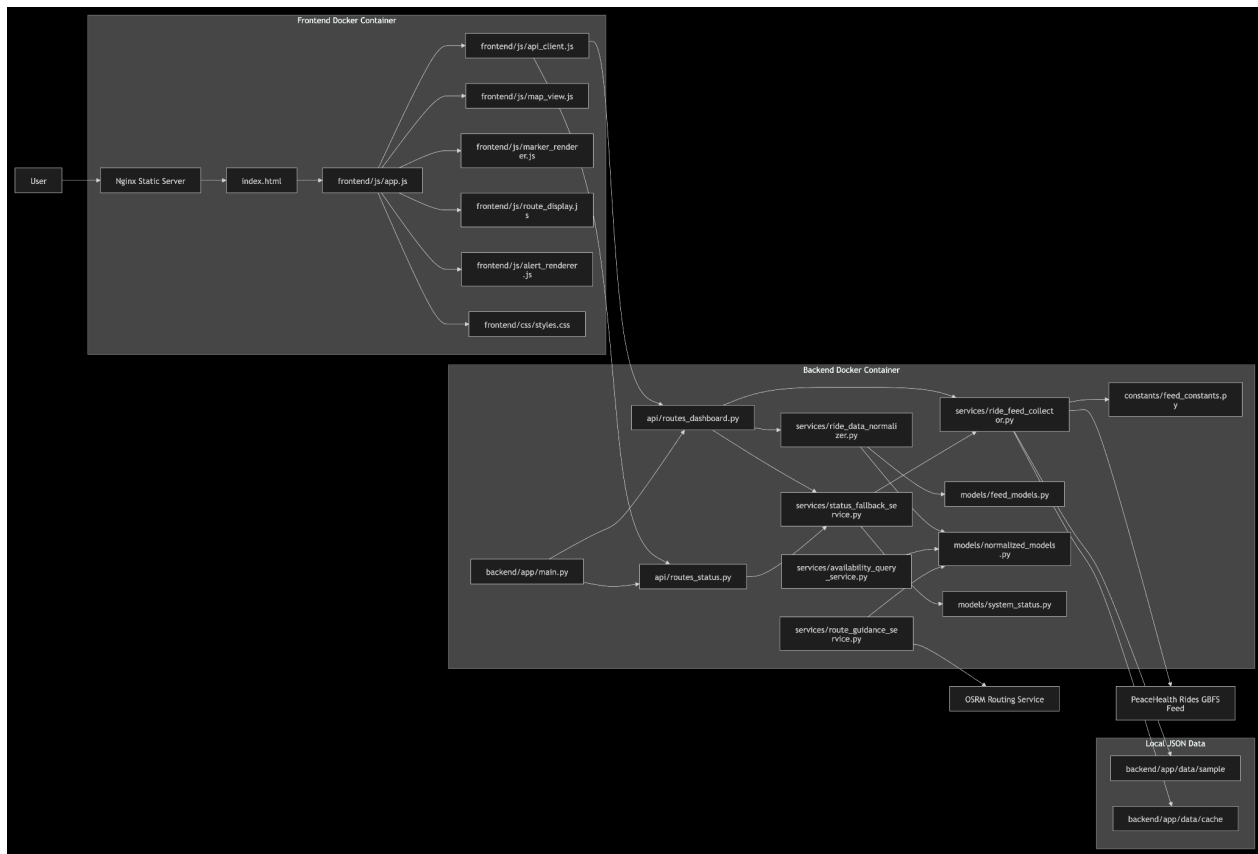
The PeaceHealth Rides Availability and Navigation Dashboard is a browser-based system that displays public PeaceHealth Rides bike and hub availability and helps users find directions to nearby bikes or hubs. The system is not intended to replace the official PeaceHealth Rides app. It does not create accounts, process payments, reserve bikes, unlock bikes, or manage private user data. Its purpose is to use public bike-share data to help users determine whether a bike or hub is nearby before they use the official rental system.

The system uses public bike-share data published through the General Bikeshare Feed Specification, or GBFS. GBFS is an open data standard for shared mobility feeds, and the GBFS documentation defines files and fields used for shared mobility systems, including station and vehicle availability data [5]. The design assumes the system will use GBFS JSON files, normalize the data into simpler project-specific objects, and expose those objects to a browser-based map.

The major services provided by the system are:

1. Retrieve public GBFS feed data
2. Normalize bike, hub, and alert data into a consistent internal structure
3. Display bikes and hubs on an interactive map
4. Let a user select a starting location and destination
5. Compute nearby bikes or hubs from the selected starting point
6. Provide route or direction information to a selected bike or hub
7. Show whether the data is live, cached, or sample data
8. Provide a fallback demonstration mode using local sample JSON

The high-level structure is shown on the next page.



1

Software Architecture

The system uses a layered modular architecture. Each module is responsible for one main task so that changes in one part of the system do not unnecessarily affect other parts. This improves organization, testing, and parallel development. The architecture separates frontend map visualization from backend data retrieval, normalization, routing, and availability services.

Overall File Structure

Project-2-PeaceHealth/

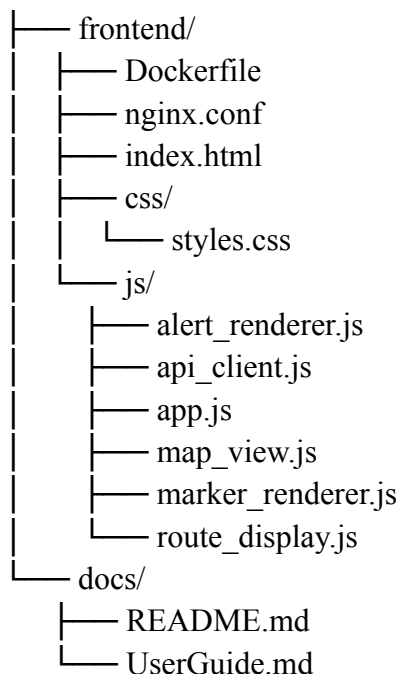
```

|— .gitignore
|— docker-compose.yml
|— backend/
|   |— Dockerfile
|   |— requirements.txt
|   |— app/

```

¹ Figure 1. A software architecture diagram showing the major modules of the PeaceHealth Rides Availability and Navigation Dashboard and their relationships

```
├── main.py
├── api/
│   ├── routes_dashboard.py
│   └── routes_status.py
├── constants/
│   └── feed_constants.py
├── data/
│   ├── cache/
│   │   ├── .gitkeep
│   │   ├── gbfs.json
│   │   ├── free_bike_status.json
│   │   ├── station_information.json
│   │   ├── station_status.json
│   │   └── system_alerts.json
│   └── sample/
│       ├── gbfs.json
│       ├── free_bike_status.json
│       ├── station_information.json
│       ├── station_status.json
│       └── system_alerts.json
├── models/
│   ├── feed_models.py
│   ├── normalized_models.py
│   └── system_status.py
├── services/
│   ├── availability_query_service.py
│   ├── ride_data_normalizer.py
│   ├── ride_feed_collector.py
│   ├── route_guidance_service.py
│   └── status_fallback_service.py
├── utils/
│   ├── distance.py
│   ├── json_loader.py
│   └── validation.py
└── tests/
    ├── fixtures/
    │   └── gbfs_samples.py
    ├── test_data_normalizer.py
    ├── test_feed_collector.py
    └── test_status_fallback_service.py
```



2

Frontend Architecture Planning

Bike Finder Map

The Bike Finder Map is the browser-based map display. It shows bike markers, hub markers, selected destination details, route information, alerts, and data freshness messages. The map acts as the main user interface for interacting with the system.

Leaflet

Leaflet is a reasonable map library for this project because it is an open-source JavaScript library designed for lightweight, mobile-friendly interactive maps [10]. Leaflet simplifies marker rendering, map interaction, and layer management while remaining easy to integrate into a browser-based application.

OpenStreetMap

The project also uses OpenStreetMap as its map tile and geographic data source. OpenStreetMap is sensible for this project because it is open-source, publicly accessible, and widely supported by mapping tools such as Leaflet. It provides the geographic map data needed to display roads, paths, and bike hub locations without requiring proprietary mapping services.

² Figure 2. Overall file structure of the system

Frontend Responsibilities

The frontend is responsible for:

- Rendering bike and hub markers on the map
- Displaying route information and estimated distances
- Displaying alerts and data freshness messages
- Allowing users to select bikes or hubs
- Requesting simplified data from the backend API
- Updating the map when new data becomes available

The frontend does not directly process raw GBFS feed files. Instead, it communicates only with the Dashboard API, which hides the raw feed structure from the user interface.

Frontend Architecture Rationale

The frontend architecture separates display logic from data retrieval and processing logic. This allows the map interface to remain stable even if the underlying feed structure changes.

The frontend design also supports modular testing. The map interface can be tested using fixed API responses without requiring live network access or live GBFS feeds.

Additionally, separating frontend responsibilities allows one team member to work on map interaction and visualization while another works on backend services and feed processing.

Backend Architecture Planning

Ride Feed Collector

The Ride Feed Collector retrieves GBFS JSON files from the public PeaceHealth Rides feed or loads local sample JSON files when the system is running in demonstration mode. This component is responsible for network access, timeout handling, JSON decoding, and returning raw feed data to the normalizer.

Ride Data Normalizer

The Ride Data Normalizer converts GBFS feed files into internal project objects. GBFS is useful as an external standard, but the map and availability logic should not need to understand the full external feed structure.

The normalizer produces simplified objects such as:

- BikeLocation
- HubLocation

- HubAvailability
- SystemAlert

Availability Query Service

The Availability Query Service provides project-specific availability operations. It answers questions such as:

- “Which bikes are currently visible?”
- “Which hubs have bikes available?”
- “Which bikes or hubs are nearest to this coordinate?”

This module also performs distance calculations.

Route Guidance Service

The Route Guidance Service produces walking route information or distance estimates between a user-selected starting point and a selected bike or hub.

OSRM (Open Source Routing Machine)

The project may use the Open Source Routing Machine (OSRM) to generate route guidance and distance estimates. OSRM is a reasonable choice for this project because it is an open-source routing engine designed for fast geographic route calculations using OpenStreetMap road data. This allows the system to provide realistic walking directions instead of relying only on straight-line distance calculations.

The first version may still use straight-line distance estimates or simplified routing depending on project constraints, while later versions may expand OSRM integration for more detailed navigation behavior.

Status and Fallback Service

The Status and Fallback Service tracks whether the system is using live data, cached data, or sample data. It records:

- Failed feed refreshes
- Missing feed files
- Invalid data
- Stale update times

This module exists so that the frontend map does not silently display misleading data.

Dashboard API

The Dashboard API exposes simplified endpoints used by the Bike Finder Map. FastAPI is a reasonable choice for this project because it is a modern Python framework for building APIs using Python type hints [11].

The API hides the raw GBFS format from the frontend and provides simplified project-specific responses.

Local Demonstration Data Store

The Local Demonstration Data Store contains sample GBFS-like JSON and optional cached responses. This is not a remote database service. It is a local file or SQLite-based storage area used to support testing and demonstration when live network access is unavailable.

Backend Responsibilities

The backend is responsible for:

- Retrieving and decoding GBFS feeds
- Normalizing raw feed data into project-specific objects
- Performing availability and distance calculations
- Providing routing or distance estimate information
- Tracking feed freshness and failure states
- Serving simplified API endpoints to the frontend
- Supporting demonstration mode with local sample data

Backend Architecture Rationale

The backend architecture separates external data concerns from frontend display concerns. GBFS data may contain more fields than the project needs, and feed structures may differ between versions. By isolating feed parsing in the Ride Feed Collector and Ride Data Normalizer, the frontend map can work with stable project-specific objects.

The backend design also supports parallel development. One teammate can implement GBFS retrieval and sample data handling while another implements route and distance logic.

The modular backend structure supports testing:

- The normalizer can be tested with sample JSON
- The availability service can be tested with fixed coordinates
- The status service can be tested with simulated feed failures
- The API can be tested independently from the frontend

Finally, the backend supports installation and demonstration constraints. If live data access fails, the Local Demonstration Data Store and Status and Fallback Service allow the system to continue demonstrating its main behavior without incorrectly presenting stale or sample data as live data.

Technology Choices

Area	Proposed Technology	Rationale
API framework	Python FastAPI	Provides typed API routes and JSON responses [11]
Map display	Leaflet with HTML / CSS / JavaScript	Lightweight open-source browser map library [10]
Data format	JSON	GBFS feeds are JSON-based [5]
Sample / cached storage	Local JSON files or SQLite	Keeps evaluation local and avoids account creation
Testing	Python unit tests	Allows parser, distance, and API behavior to be tested
Installation	Docker	Matches course installation expectations [9]

Alternative Designs Considered

Alt-1: Direct Browser-to-GBFS design

One alternative was to have the browser fetch the GBFS feed directly and render the data without a local API. This would reduce the amount of backend code. However, it would push feed parsing, error handling, normalization, and fallback logic into the browser. That would make the map module more complex and harder to test. This design was rejected because the project benefits from a separate normalization and availability layer.

Alt-2: Full Database-driven design

Another alternative was to store all bike and hub updates in a local database and have the dashboard query only the database. This could support history, trends, and availability over time.

However, the initial project does not require historical analytics. A full database-first design would add complexity and might distract from the core use case of showing current availability.

Alt-3: Static data only design

A third alternative was to only create a map for the various bike hubs around Eugene. This was rejected because bikes are not only found at these hubs, and sometimes the hubs can be empty. As such, the final design displays both hubs and bikes in separate pins.

Software Modules

Ride Feed Collector

Role and Primary Function

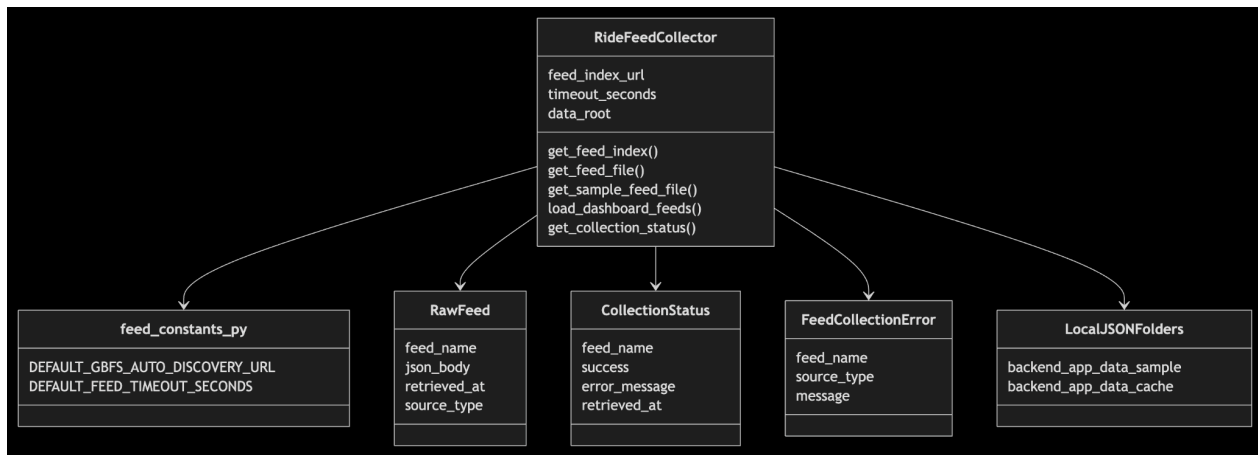
The Ride Feed Collector obtains raw GBFS JSON data. It can operate in live mode, where it retrieves data from the public feed, or sample mode, where it loads local JSON files. It does not decide how bikes or hubs should be displayed. It only retrieves raw data and reports whether retrieval succeeded. It is implemented in `backend/app/services/ride_feed_collector.py`

Interface to Other Modules

Function	Input	Output
<code>get_feed_index()</code>	None or configured feed URL	Raw GBFS feed index JSON
<code>get_feed_file(feed_name)</code>	Feed file name such as <code>free_bike_status</code> or <code>station_status</code>	Raw JSON for that feed
<code>Get_sample_feed_file(feed_name)</code>	Local sample feed name	Local sample JSON
<code>get_collection_status()</code>	None	Feed retrieval status object

Static Model

The static structure of this module is shown on the next page.

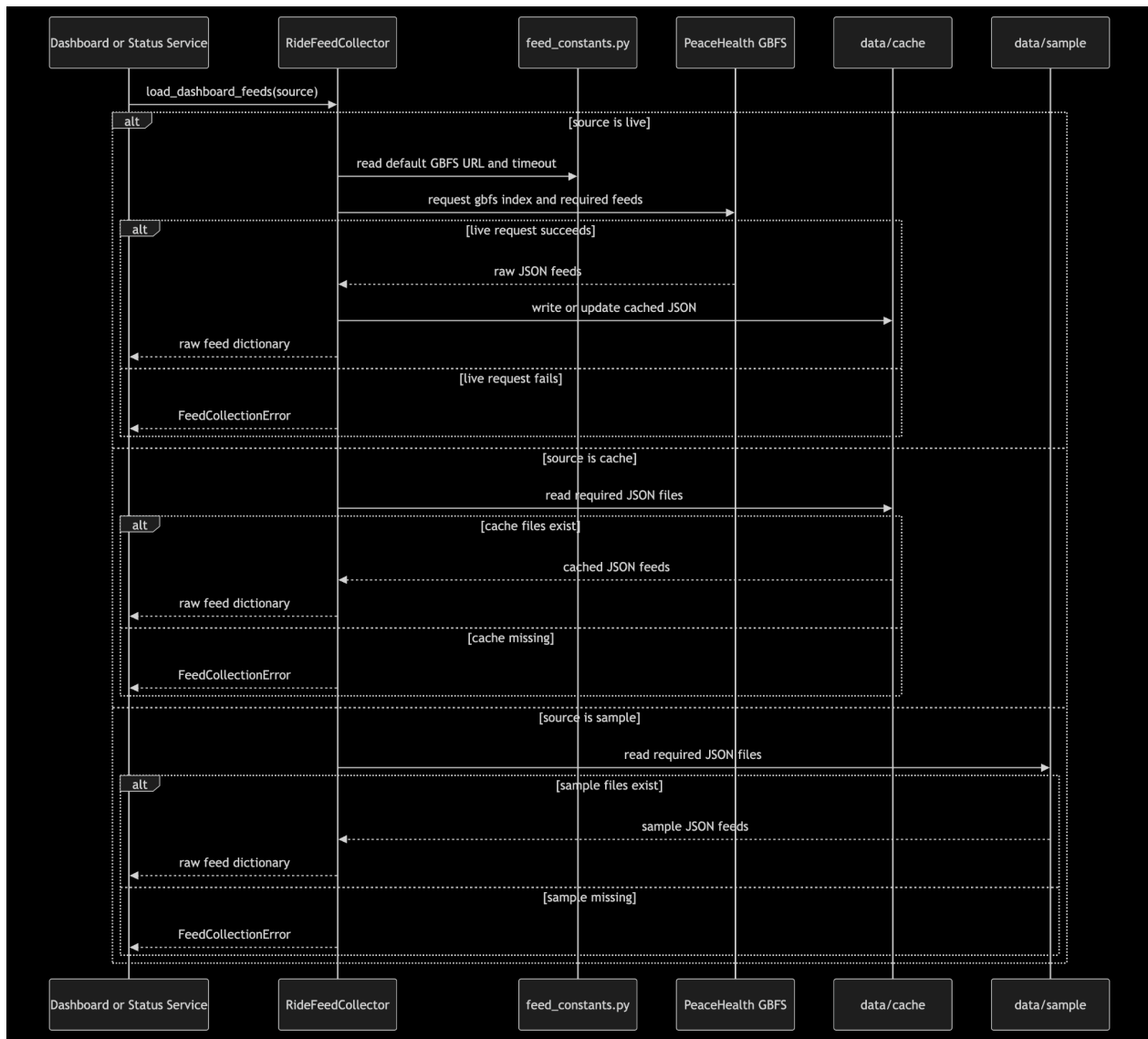


3

Dynamic Model

The dynamic behavior of feed collection during a normal data refresh is shown on the next page.

³ Figure 3. A UML-style class diagram showing the Ride Feed Collector and its related data structures



4

Design Rationale

This module exists because external data access is a separate responsibility from data interpretation. If the PeaceHealth Rides feed URL, timeout behavior, or sample file strategy changes, the Ride Feed Collector can change without requiring changes to map display code or availability ranking code.

⁴ Figure 4. A sequence diagram showing the Ride Feed Collector retrieving live GBFS data and returning it to the normalizer

Ride Data Normalizer

Role and Primary Function

The Ride Data Normalizer converts raw GBFS JSON into internal project objects. GBFS feed files may contain many fields, and different feed files describe different parts of the bike-share system. The normalizer produces a consistent internal format used by the rest of the system. It is implemented in `backend/app/services/ride_data_normalizer.py`

Interface to Other Modules

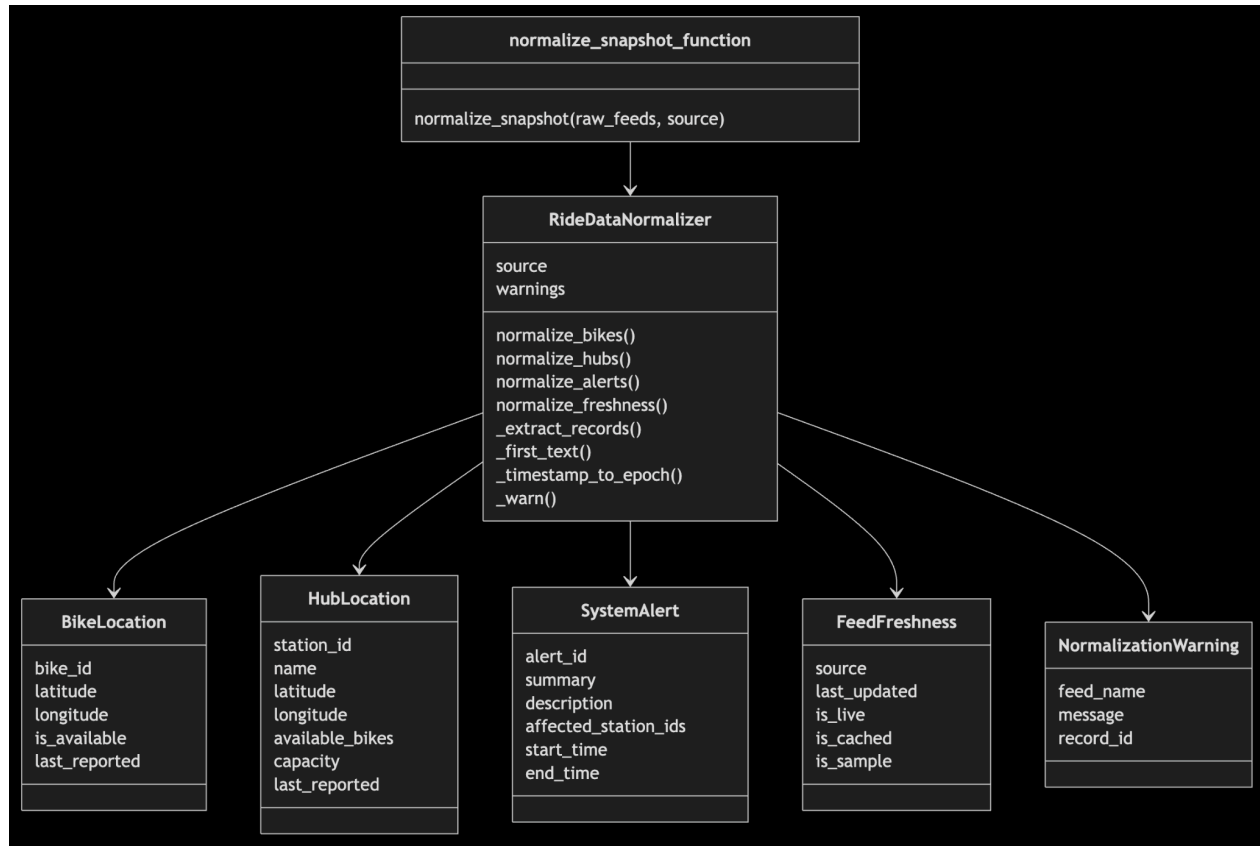
Function	Input	Output
<code>normalize_bikes(raw_free_bike_status)</code>	Raw GBFS free bike JSON	List of BikeLocation objects
<code>normalize_hubs(raw_station_information, raw_station_status)</code>	Raw station information and status JSON	List of HubLocation objects
<code>normalize_alerts(raw_system_alerts)</code>	Raw alerts JSON	List of SystemAlert objects
<code>normalize_timestamp(raw_feed)</code>	Raw feed JSON	Standard timestamp or freshness object

Internal Data Objects

Object	Main Fields
BikeLocation	Bike_id, latitude, longitude, is_available, last_reported
HubLocation	Station_id, name, latitude, longitude, available_bikes, capacity, last_reported
SystemAlert	Alert_id, summary, description, affected_station_ids, start_time, end_time
FeedFreshness	Source, last_updated, is_live, is_cached, is_sample

Static Model

The static structure of the normalized data objects is shown below

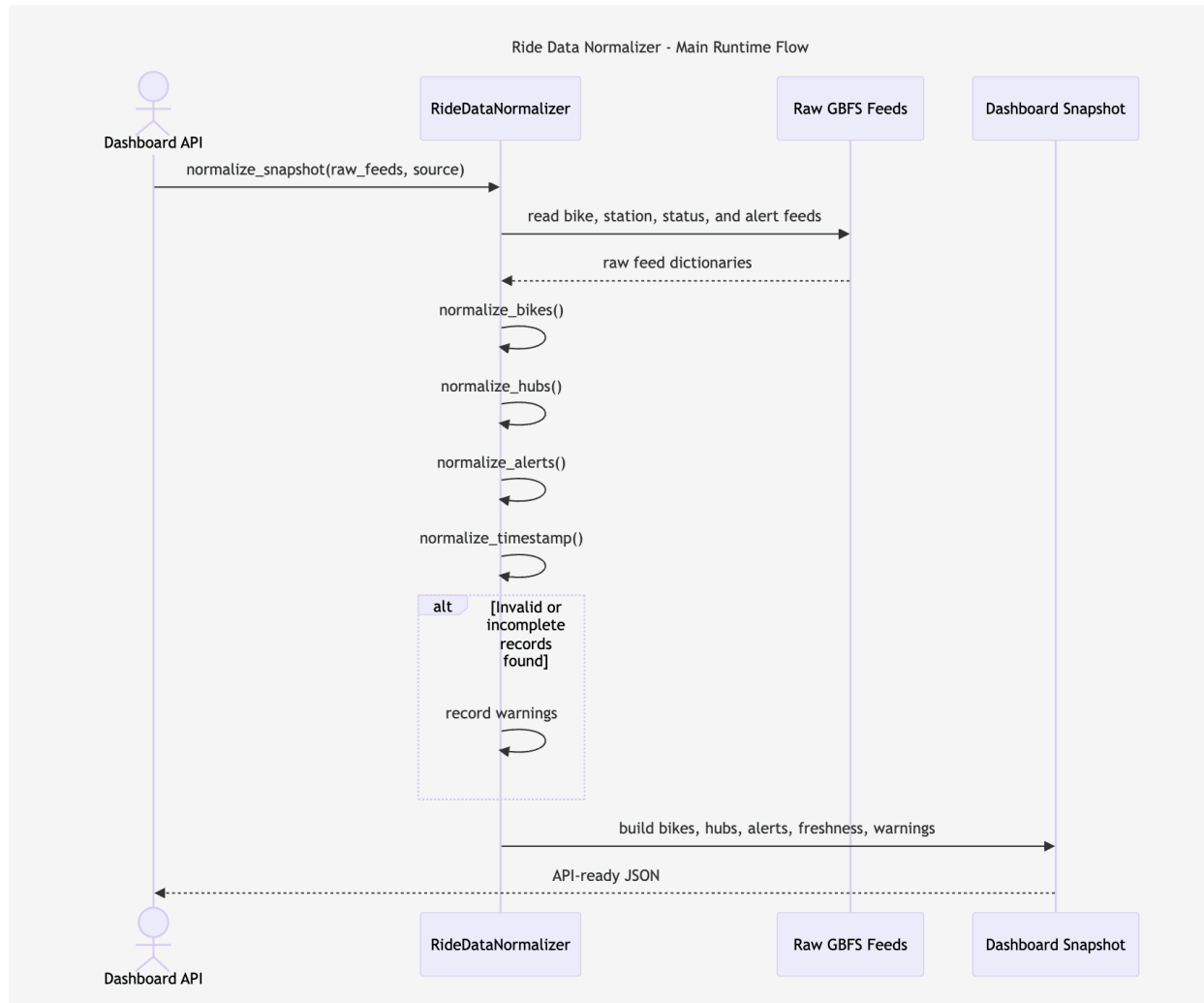


5

Dynamic Model

The dynamic behavior of normalization is included in figure 5, and normalization occurs between feed retrieval and availability query responses. Raw GBFS data is gathered and converted into the individual objects the client consumes.

⁵ Figure 5. A UML-style class diagram showing the normalized bike, hub, alert, and feed freshness objects



6

Design Rationale

The normalizer prevents the rest of the system from depending directly on GBFS field names and feed structure. This makes the dashboard easier to maintain if the project changes which feed files it uses or if the team adjusts the internal objects needed by the map.

Availability Query Service

Role and Primary Function

The Availability Query Service answers project-specific questions about available bikes and hubs. It filters unavailable entries, computes distances, ranks nearby destinations, and provides

⁶ Figure 6. A sequence diagram showing the GBFS data being converted into normalized bike, hub, alert, and feed freshness objects

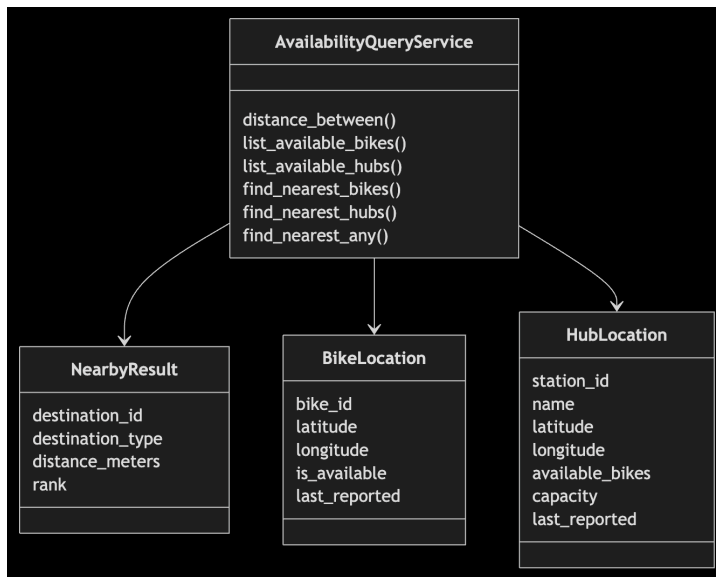
simplified results to the Dashboard API. It is implemented in `backend/app/services/availability_query_service.py`

Interface to Other Modules

Function	Input	Output
<code>list_available_bikes()</code>	Normalized bike list	Filtered bike list
<code>list_available_hubs()</code>	Normalized hub list	Filtered hub list
<code>find_nearest_bikes(latitude, longitude, limit)</code>	Starting coordinate and result limit	Ranked nearby bike list
<code>find_nearest_hubs(latitude, longitude, limit)</code>	Starting coordinate and result limit	Ranked nearby hub list
<code>Distance_between(coord_a, coord_b)</code>	Two coordinates	Distance estimate

Static Model

The relationship between normalized data objects and availability query operations is shown below.

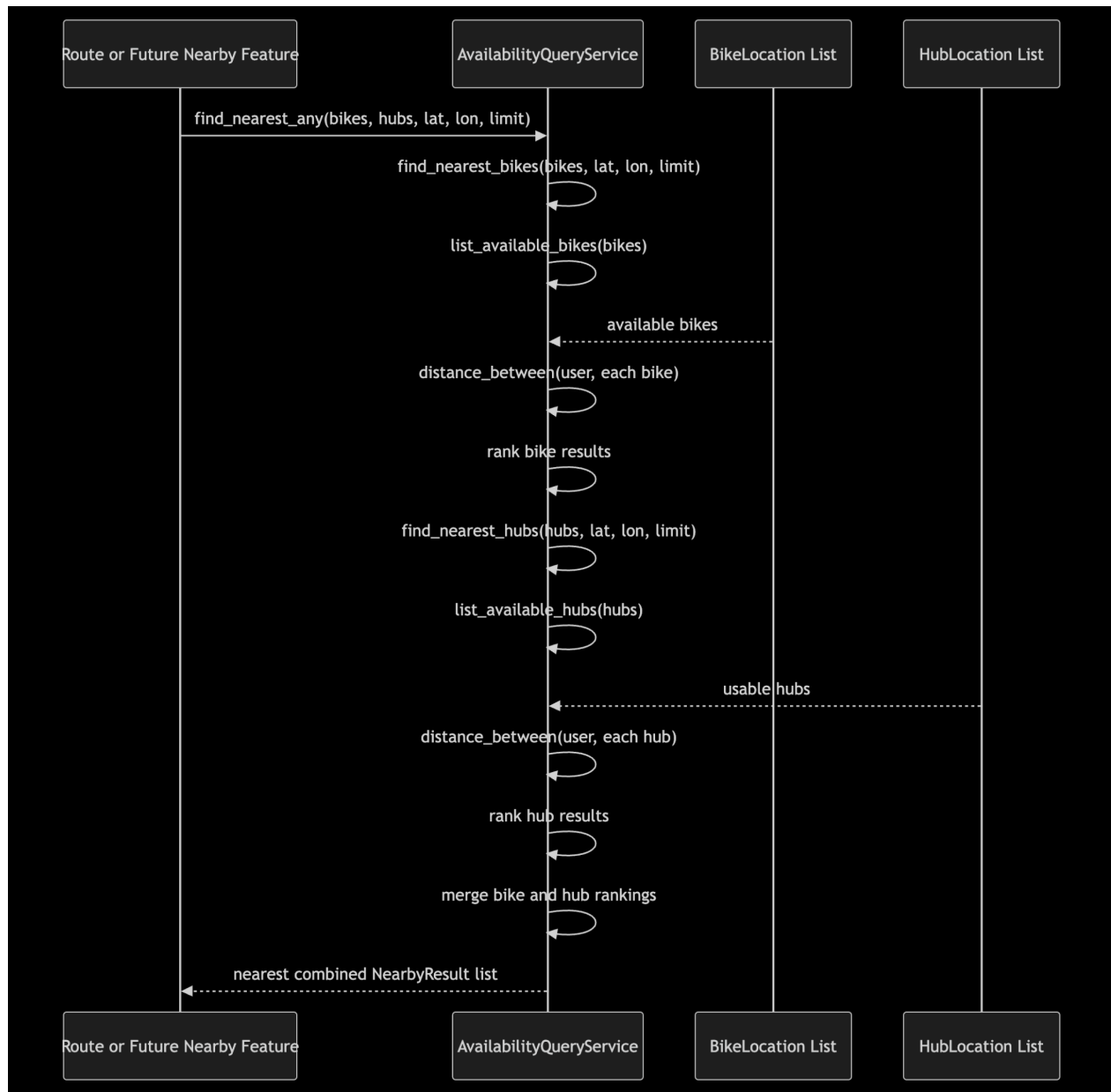


7

⁷ Figure 7. A UML-style class diagram showing the Availability Query Service and its relationship to bikes, hubs, and coordinates

Dynamic Model

The dynamic behavior for finding the nearest bike is shown below



8

Design Rationale

This module exists because distance calculations and ranking logic are not display logic. The map should not need to know how the system computes nearest bikes. Keeping availability queries separate makes the project easier to test with known coordinate examples.

⁸ Figure 8. A sequence diagram showing the user selecting a starting location and the system returning nearby bikes

Route Guidance Service

Role and Primary Function

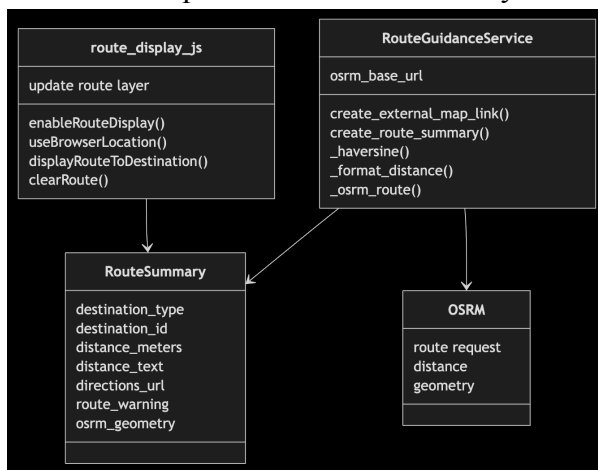
The Route Guidance Service provides directions or route estimates from a user-selected starting location to a selected bike or hub. In the initial system, this may be implemented as an approximate walking distance and an external map link. It is implemented in `backend/app/services/route_guidance_service.py` and frontend `route_display.js`

Interface to Other Modules

Function	Input	Output
<code>create_route_to_bike(start_coord, bike_id)</code>	Starting coordinate and selected bike	Route summary
<code>create_route_to_hub(start_coord, station_id)</code>	Starting coordinate and selected hub	Route summary
<code>create_external_map_link(start_coord, destination_coord)</code>	Starting and destination coordinates	Browser-safe URL
<code>estimate_walking_distance(start_coord, destination_coord)</code>	Starting and destination coordinations	Distance estimate

Static Model

The Route Guidance Service uses the selected `BikeLocation` or `HubLocation` and a user-provided coordinate to produce a `RouteSummary`. This relationship is shown below.

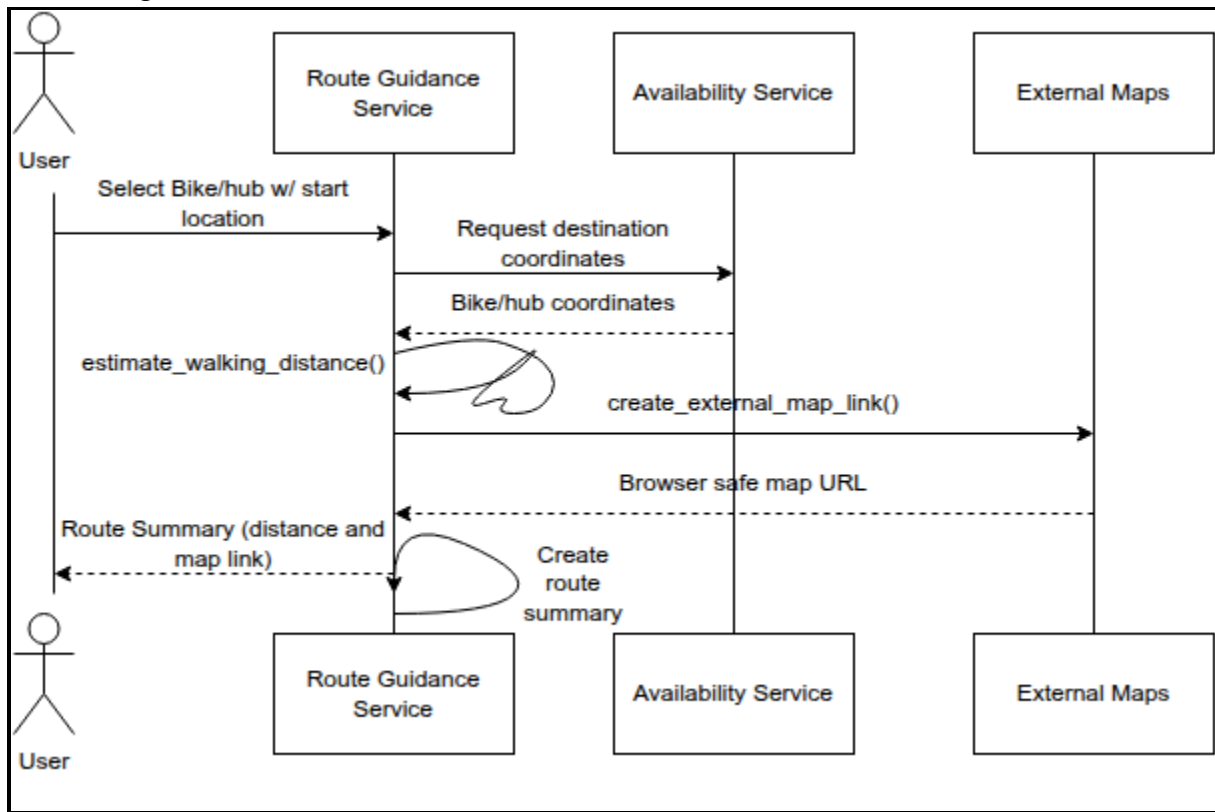


9

⁹ Figure 9. A UML-style class diagram showing the Route Guidance Service and RouteSummary object

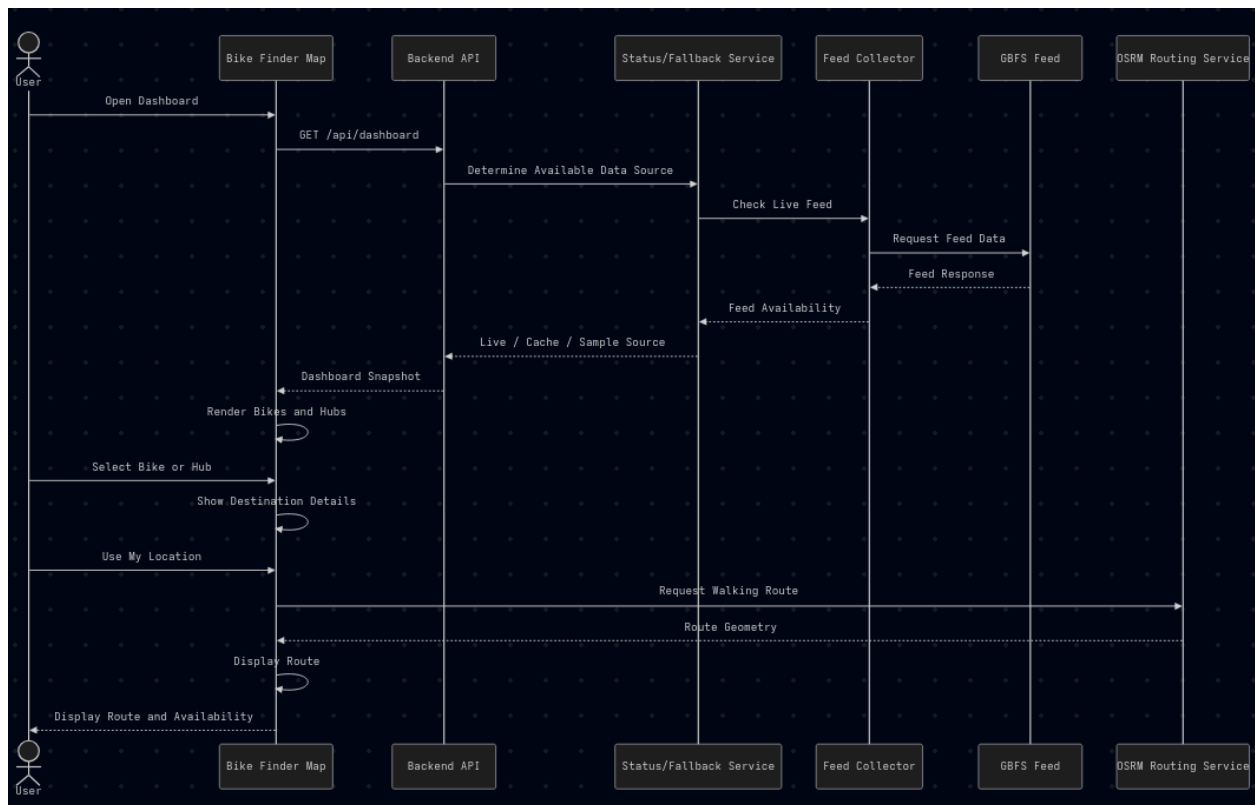
Dynamic Model

The route generation scenario is shown below



¹⁰

¹⁰ Figure 10. A UML sequence diagram showing route guidance service steps



Design Rationale

This module exists because routing is likely to evolve. The project may begin with a simple direction link or approximate distance. Later, the team may add route polylines or step-by-step directions. Keeping routing separate prevents those changes from disrupting feed parsing or availability queries.

Status and Fallback Service

Role and Primary Function

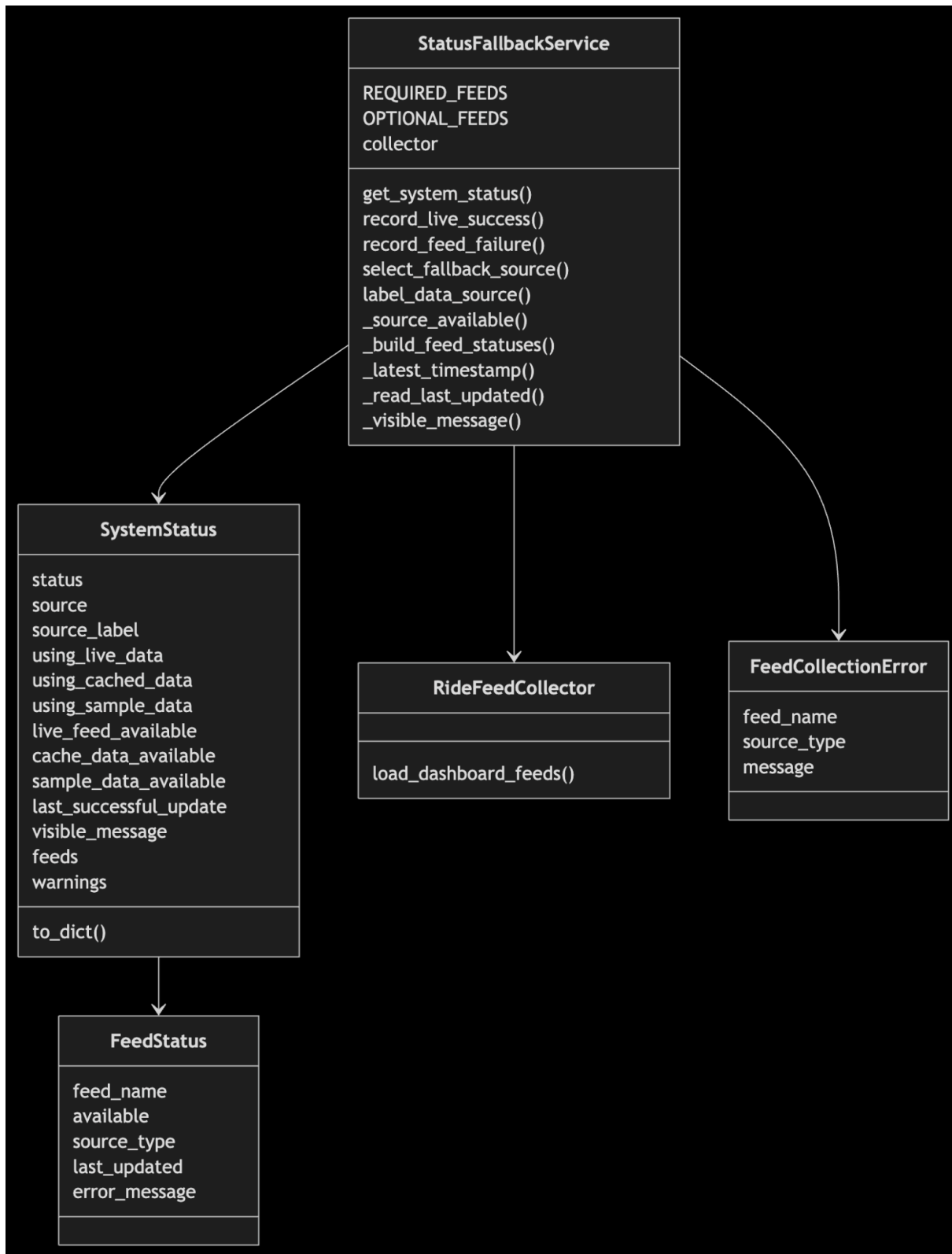
The Status and Fallback Service tracks whether the system is currently using live data, cached data, or sample data. It also records feed failures, invalid JSON, stale timestamps, and missing data. The purpose of this module is to keep the system honest about the quality of the displayed information. It is implemented in `backend/app/services/status_fallback_service.py` and frontend `route_display.js`

Interface to Other Modules

Function	Input	Output
<code>record_live_success(feed_name, timestamp)</code>	Feed name and timestamp	Updated status
<code>record_feed_failure(feed_name, error)</code>	Feed name and error	Updated status
<code>get_system_status()</code>	None	System status object
<code>select_fallback_source(feed_name)</code>	Feed name	Cached or sample source decision
<code>label_data_source(source_type)</code>	Live/Cached/sample	User-facing label

Static Model

The status and fallback structure is shown on the next page.

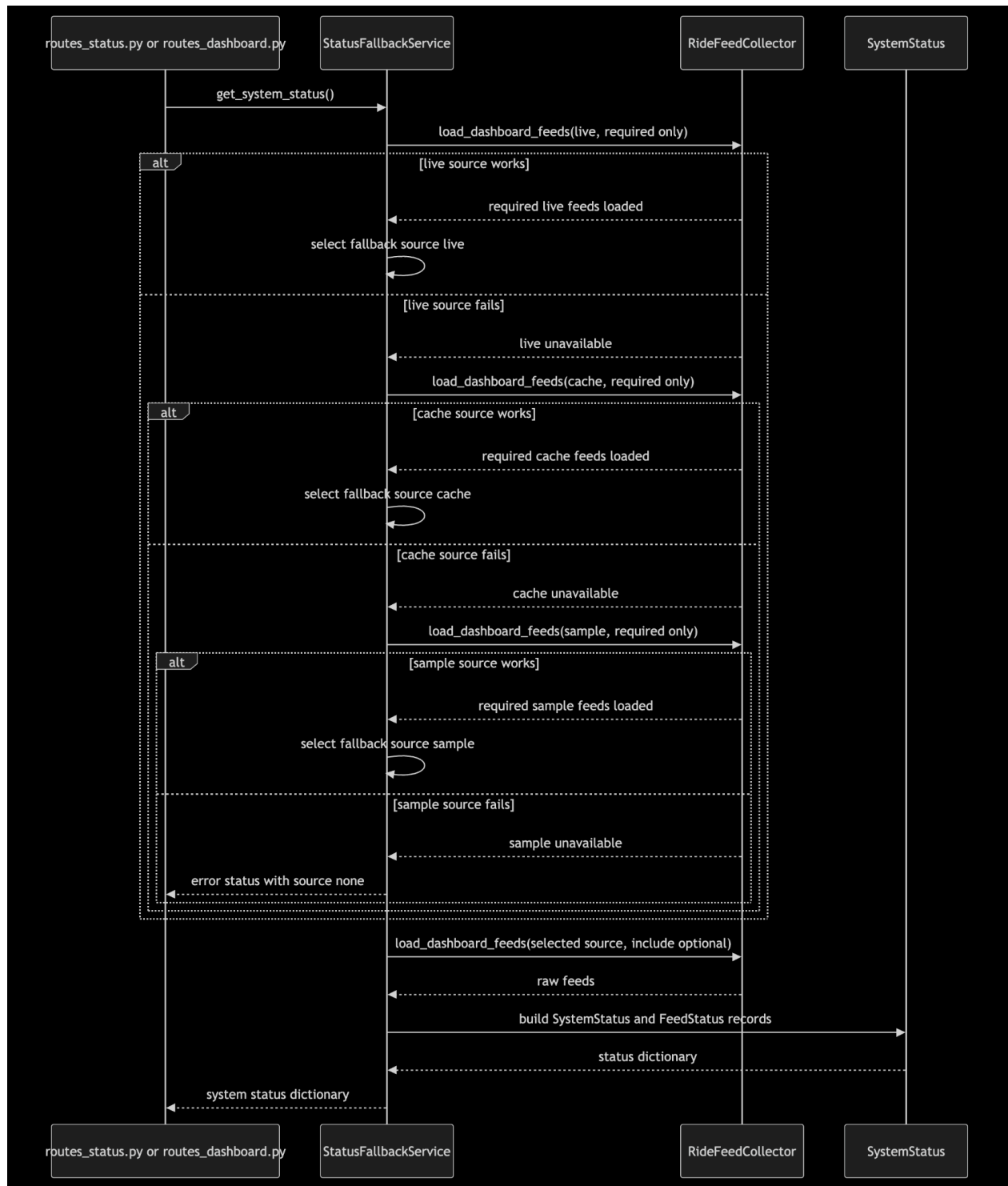


11

¹¹ Figure 11. A UML-style class diagram showing the Status and Fallback Service and its status objects.

Dynamic Model

The fallback behavior when live data fails is shown below



12

¹² Figure 12. A sequence diagram showing how the system handles live feed failure by returning cached or sample data with a warning

Design Rationale

This module exists because stale transportation data can mislead users. If the system displays sample or cached data, the user should know. This module centralizes the status labels and fallback decisions instead of scattering them through the map and feed code.

Dashboard API

Role and Primary Function

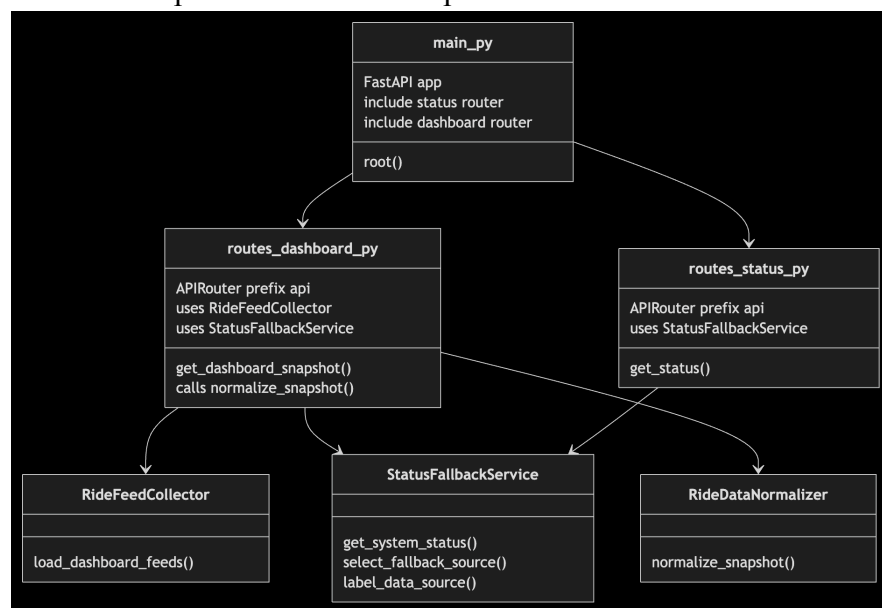
The Dashboard API exposes two primary endpoints: `/api/status` for system status and `/api/dashboard` for a combined dashboard snapshot containing normalized bikes, hubs, alerts, freshness metadata, and status information.

Interface to Other Modules

Endpoint	Purpose
GET <code>/api/dashboard</code>	Return collection of data used in the app.
GET <code>api/status</code>	Return live/cached/sample data status

Static Model

The API endpoints and service dependencies are shown below

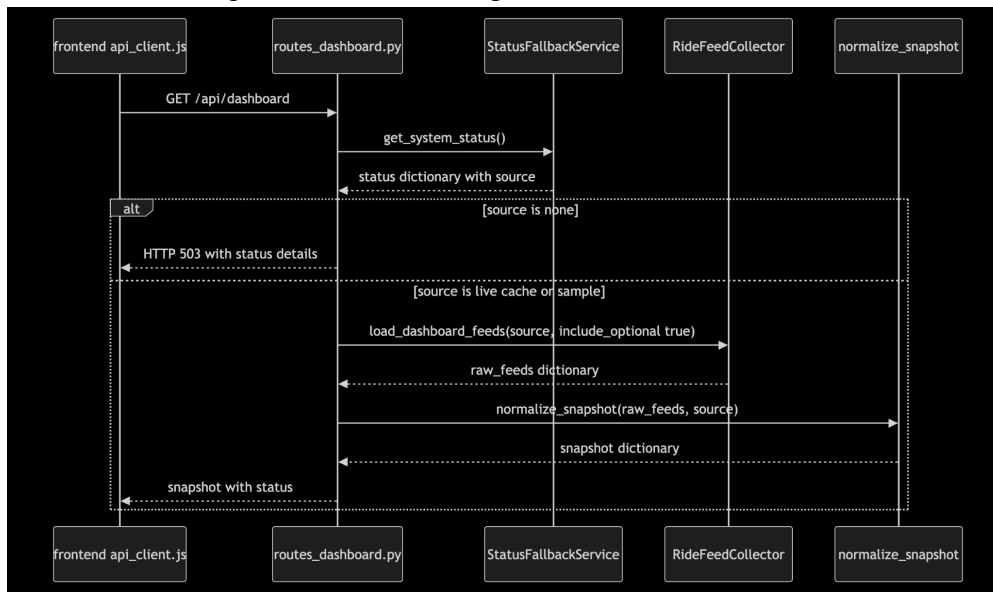


13

¹³ Figure 13. A component diagram showing Dashboard API routes and the services that provide each response.

Dynamic Model

The main API request flow for loading the dashboard is shown below



14

Design Rationale

This module exists to create a stable boundary between browser code and system logic. The map should only need simple JSON responses. It should not need to know how GBFS files are retrieved, parsed, cached, or validated.

Bike Finder Map

Role and Primary Function

The Bike Finder Map is the browser-facing module. It displays the map, markers, filters, selected destination details, route information, and data status. It communicates with the Dashboard API and renders the returned JSON. It is implemented in the following files within the frontend/js/~ folder: map_view.js, marker_renderer.js, app.js, alert_render.js, and route_display.js.

Interface to Other Modules

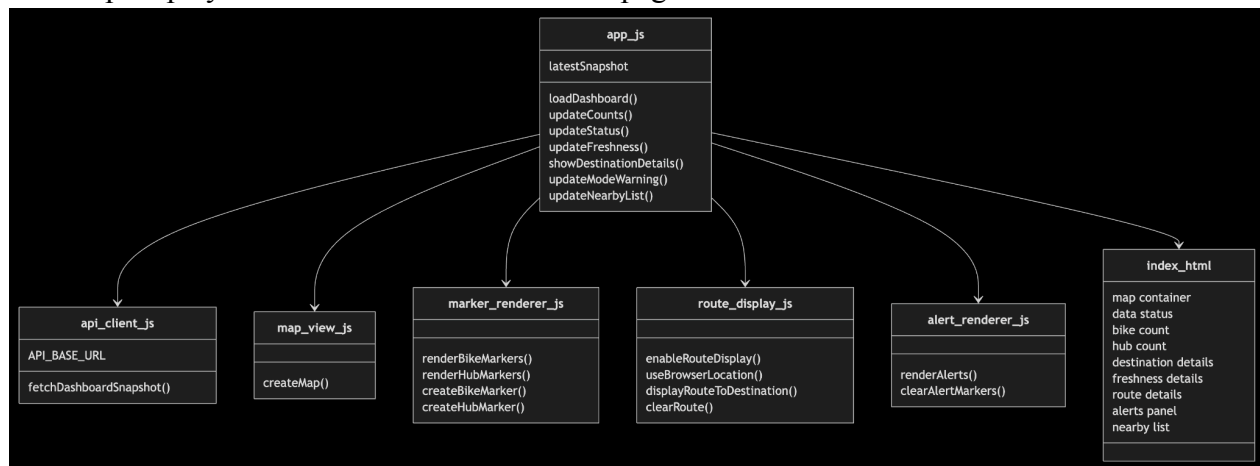
Function	Input	Output
localDashboardData()	None	Requests bikes, hubs, alerts, and status

¹⁴ Figure 14. A sequence diagram showing the dashboard map requesting bikes, hubs, alerts, and status from the API.

renderBikeMarkers(bikes)	Bike list	Bike markers on map
renderHubMarkers(hubs)	Hub list	Hub markers on map
selectStartingLocation(location)	Coordinate or address	Stored start point
selectDestination(marker)	Bike or hub marker	Destination detail panel
displayRoute(routeSummary)	Route summary	Route display
displayStatus(status)	System status	Data freshness message

Static Model

The map display structure is shown on the next page.

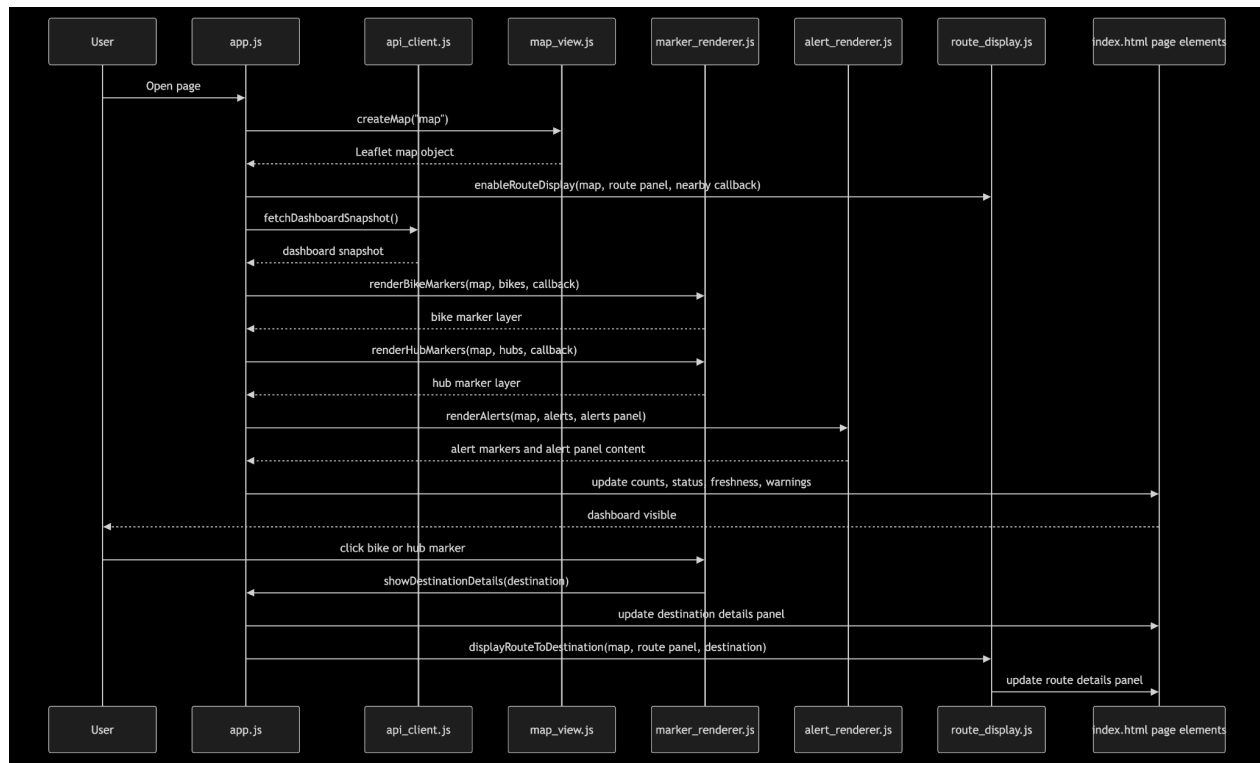


15

Dynamic Model

The map loading and marker rendering flow is shown on the next page.

¹⁵ Figure 15. A UML-style class diagram showing the browser map objects and their relationship to API response objects



16

Design Rationale

This module exists as the user-facing portion of the system, but it is intentionally kept thin. It should request data, render data, and react to user selections. It should not contain the core GBFS parsing or availability ranking rules.

Local Demonstration Data Store

Role and Primary Function

The Local Demonstration Data Store contains sample JSON and optional cached live responses. It allows the project to run without a live network dependency during grading or demonstration. It is implemented in `backend/app/services/ride_feed_collector.py`, `backend/app/data/sample/`, and `backend/app/data/cache`

Interface to Other Modules

¹⁶ Figure 16. Dynamic model for the current Bike Finder Map module without cross-referencing the broader dashboard sequence.

Function	Input	Output
<code>read_sample(feed_name)</code>	Feed name	Sample JSON
<code>write_cache(feed_name, raw_json)</code>	Feed name and JSON	Stored cache
<code>read_cache(feed_name)</code>	Feed name	Cached JSON
<code>cache_exists(feed_name)</code>	Feed name	Boolean

Static Model

The Local Demonstration Data Store appears in Figure 1 and Figure 9 as a dependency of fallback behavior.

Dynamic Model

The cache/sample fallback behavior is shown in Figure 10.

Design Rationale

This module exists to make the system demonstrable and testable. It also reduces the risk that a temporary external feed issue prevents the team from showing the system's main functions.

Dynamic Models of Operational Scenarios

The major operational scenarios for this project are as follows:

1. Load the dashboard with live bike and hub data
2. Find the nearest available bike
3. Select a bike or hub and request directions
4. Handle live feed failure using cached or sample data

These scenarios are represented by Figure 3, Figure 6, Figure 8, Figure 10, and Figure 12.

Project Plan

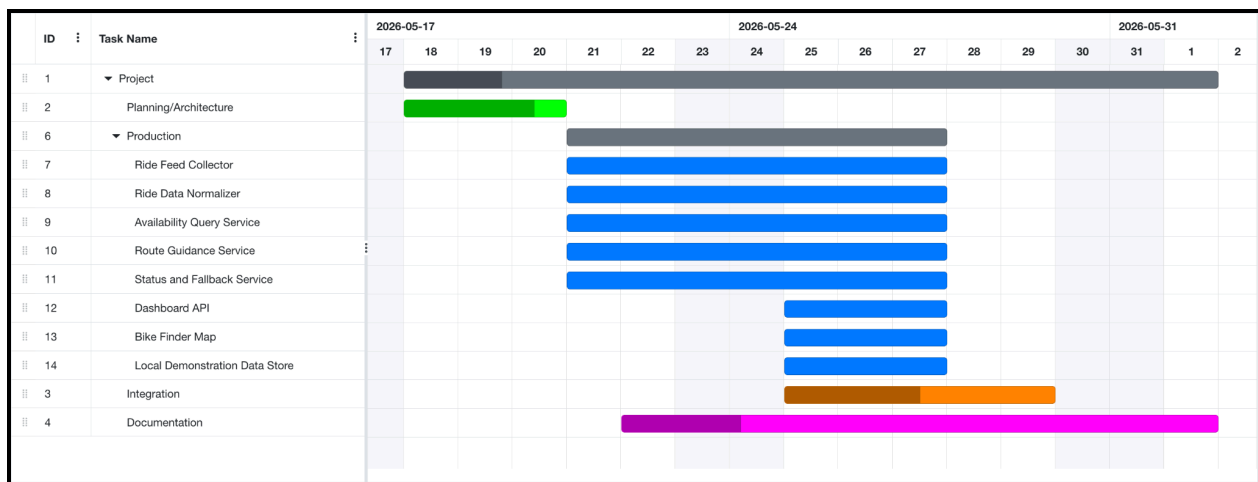
Management

Our group focused on a management system with Peyton as the team lead and making sure the project stayed on schedule. Choices made as a team would be discussed in the meetings, and if there were stalemates Peyton as team lead would be the deciding vote. Individual modules are assigned to the people below:

- Ride Feed Collector: Drew
- Ride Data Normalizer: Drew
- Availability Query Service: Jack
- Route Guidance Service: Jack
- Status and Fallback Service: Dacian
- Dashboard API: Peyton
- Bike Finder Map: Dacian
- Local Demonstration Data Store: Peyton

Reporting / Communication

Communication between members is done primarily through text, the main google document, and in-person meetings. Meetings were done twice a week after class, and meeting notes kept track of who attended, what was discussed, when and where we met, what was accomplished, and any additional notes needed. For example, the last meeting was on June 1 in the EMU, and we talked about the rewrite of some parts of this document, and that later we need to go through and refactor the code to include comments.



17

¹⁷ Figure 17. Gantt chart of project plan / timeline

The Gantt chart in Figure 17 shows the planned timeline for the project, including the initial planning and architecture phase, parallel production work on the backend and frontend modules, integration work, and documentation. The chart also shows that several production modules are scheduled to proceed in parallel after the core planning tasks are complete.

Planning and Architecture

The Planning/Architecture phase runs first and focuses on finalizing the system requirements, design structure, and implementation choices, beginning with in-person interviews with experts to guide the system goal. During this phase, the team will conduct in-person interviews, finalize the SRS and SDS, choose the backend framework and map technology, verify the GBFS feed structure, define the API response formats, and set up the Docker and repository structure.

Following initial ideation, insight collected from a series of interviews with experts in respective fields will be used to inspire and refine our system plan. This step is critical to complete before implementation, as we will be looking for technical problems in demand of solutions, applicable to the UO campus. In the event that interviews do not take place before May 21st, subsequent task timelines are pushed back respectively.

This phase also establishes the boundaries between the major modules. The Ride Feed Collector will be responsible for retrieving raw GBFS data, the Ride Data Normalizer will convert that data into internal objects, the Dashboard API will expose simplified endpoints, and the Bike Finder Map will consume those endpoints in the browser.

Core Structure Production

The first production milestone is to produce a working core structure for the system. Backend work will include a GBFS fetcher, a basic parser, `/api/bikes` and `/api/hubs` endpoints, and a sample JSON mode. Frontend work will include the base HTML, CSS, and JavaScript setup, Leaflet/OpenStreetMap map integration, a map centered on Eugene, and hard-coded markers for early testing.

The first integration target is for the frontend to fetch live backend data and display markers created from real API responses by the end of May 27.

Parallel Feature Development

After the core structure is working, the team will move into parallel feature development. Backend work will include the alerts endpoint, nearest-distance calculation, and error handling. Frontend work will include marker styling, visual distinction between bikes and hubs, click selection panels, alert display, start-location input, and nearby bike display.

This stage connects directly to the SDS modules. Jack's Availability Query Service will support nearest-bike and nearest-hub calculations. Jack's Route Guidance Service will support distance

estimates, walking-route approximation, route display, and destination highlighting. Dacian's Status and Fallback Service will handle failed feed states and data-source labels. Dacian's Bike Finder Map will display the user-facing results. Peyton's Dashboard API will provide the endpoints needed by the browser interface, while the Local Demonstration Data Store will provide sample data for testing and demo reliability.

Directions and Routing

The routing portion of the project will support a user-selected starting location, distance calculation, walking-route approximation, route display on the map, and destination highlighting. If full routing becomes too difficult within the project timeline, the fallback plan is to provide a distance estimate, a straight-line path, or a message such as "walk approximately X minutes." This fallback keeps the system aligned with the requirements while reducing the risk that routing complexity prevents the rest of the dashboard from working.

Integration and Testing

The integration phase will focus on making the modules work together as a complete system. The team will fix API and frontend mismatches, test the interface in a browser, test the system with Docker, verify that the demo works on another machine, remove placeholder UI, and fix remaining bugs.

This phase is especially important because the system depends on several connected pieces: live or sample GBFS data, backend normalization, API responses, frontend rendering, route display, and fallback behavior.

Documentation

Documentation will continue alongside implementation and extend into the final project phase. Documentation tasks include final SRS and SDS cleanup, user documentation, installation instructions, the README, the demo script, and presentation slides.

This documentation is necessary so that evaluators can install, run, and understand the system without needing accounts, external setup, or undocumented assumptions.

Risks and Risk Mitigations

The project has several risks that could affect completion, demonstration, or correctness. The team will monitor these risks during twice-weekly meetings and through individual module updates.

Risk 1: Live GBFS feed becomes unavailable or changes format

Possible Impact

The dashboard may fail to load live bike or hub data during testing or demonstration

Likelihood

Medium

Mitigation Strategy

Maintain sample JSON files and cached feed responses. The SStatus and Fallback Service will label the data as live, cached, or sample

Responsible Role

Ride Feed Collector / Status and Fallback Service

Risk 2: Routing implementation is more difficult than expected

Possible Impact

Users may not receive full walking directions by the deadline

Likelihood

Medium

Mitigation Strategy

Implement route guidance in levels: first straight-line distance, then external route link, then full route link if time allows

Responsible Role

Route Guidance Service

Risk 3: Frontend and Backend response formats do not match

Possible Impact

Map markers, detail panels, or nearby lists may fail even if backend logic works

Likelihood

Low

Mitigation Strategy

Define API response shapes early and test endpoints with sample JSON before connecting frontend components

Responsible Role

Dashboard API / Bike Finder Map

Risk 4: Docker setup fails on a teammate's or evaluator's machine

Possible Impact

Program would be difficult to install / grade

Likelihood

Medium

Mitigation Strategy

Test 'docker compose up –build' on at least two different machines before submission and document any known issues in the README

Responsible Role

Team Lead / Dashboard API

Risk 5: User interview data is too limited

Possible Impact

The SRS may appear speculative instead of grounded in real user needs

Likelihood

High

Mitigation Strategy

Conduct 20 short interviews with realistic potential users and summarize findings in the SRS using aggregate tables and requirement traceability

Responsible Role

Team Lead / Documentation

Risk 5: Team members fall behind on assigned modules

Possible Impact

Integration may be delayed or incomplete

Likelihood

Medium

Mitigation Strategy

Use twice-weekly meetings to report completed work, blockers, and next tasks. Team lead makes final decisions during stalemates

Responsible Role

Team lead

Risk 6: Sample data does not reflect realistic PeaceHealth Rides data

Possible Impact

Demo mode may not convincingly show intended system behavior

Likelihood

Low

Mitigation Strategy

Base sample JSON structure on GBFS feed examples and include bikes, hubs, alerts, live-data failure states, and empty-hub cases

Responsible Role

Local Demonstration Data Store

Risk 7: UI is hard to use on smaller screens

Possible Impact

Users may not be able to select bikes or view route details on mobile-sized displays.

Likelihood

Medium

Mitigation Strategy

Add measurable viewport requirements and test the interface at common phone-sized widths such as 390x844 and 360x640 pixels.

Responsible Role

Bike Finder Map

References

- [1] City of Eugene, “Bike Share | Eugene, OR Website.” Describes PeaceHealth Rides as a bike-share system with publicly available bikes, stations, and service areas. [Link](#)
- [2] PeaceHealth Rides, “Eugene Bike Share.” Describes scan-to-unlock, riding, returning, and account/app-related unlock options. [Link](#)
- [3] Apple App Store, “PeaceHealth Rides.” Describes official app functions including finding bicycles, creating an account, paying, reserving bicycles, managing an account, and contacting support. [Link](#)
- [4] University of Oregon Transportation Services, “PeaceHealth Rides Bike Share Program.” Describes UO-specific PeaceHealth Rides pricing options for students, faculty, and staff. [Link](#)
- [5] GBFS.org, “General Bikeshare Feed Specification Reference.” Describes the types of files and fields used in GBFS feeds. [Link](#)
- [6] GBFS.org, “General Bikeshare Feed Specification Home.” Describes GBFS as an open data standard for shared mobility modes. [Link](#)
- [7] A. Hornof, “Software Requirements Specification Template,” CS 422 Software Methodologies. Used for SRS structure, ConOps sections, use cases, and requirements organization. [Link](#)
- [8] CS 422 Project Proposal Instructions. Used for project constraints regarding realistic requirements, real users, no machine learning, installation, account requirements, and local execution.
- [9] CS 422 Project 2 Guidelines. Instructions on the submissions as well as any rules regarding them. [Link](#)
- [10] Leaflet, “Leaflet — a JavaScript library for interactive maps.” Describes Leaflet as an open-source JavaScript library for mobile-friendly interactive maps. [Link](#)
- [11] FastAPI Documentation. Describes FastAPI as a Python web framework for building APIs based on Python type hints. [Link](#)

[12] A. Hornof, “Software Design Specification Template,” CS 422 Software Methodologies. Used for SDS structure, architecture expectations, module descriptions, design rationale, alternative designs, and dynamic model expectations. [Link](#)

[13] Uvicorn, “Uvicorn — An ASGI web server, for Python.” Describes Uvicorn as an ASGI web server implementation for Python

[14] OSRM. A modern C++ routing engine for shortest paths in road networks. [Link](#)

[15] Interview format

Acknowledgements

The SRS was drafted using the CS 422 SRS template structure and the CS 422 project instructions. The system idea was developed as a public-information dashboard for PeaceHealth Rides availability and directions. The draft should be revised after the team conducts interviews with real potential users and confirms implementation constraints with the course staff.

The SDS was written using the CS 422 SDS template structure. The project design builds on the team’s SRS for a public PeaceHealth Rides bike and hub availability dashboard. The design should be revised as the team confirms implementation details, receives user feedback, and determines the exact routing and mapping approach.